MMKP Resource Allocation in IoT-EC-CC Networks

Generated by Doxygen 1.9.8

1 Namespace Index 1
1.1 Namespace List . . . 1

2 Hierarchical Index 3
2.1 Class Hierarchy . . . 3

3 Class Index 5
3.1 Class List . . . 5

4 File Index 7
4.1 File List . . . 7

5 Namespace Documentation 9
5.1 convert Namespace Reference . . . 9
5.1.1 Detailed Description . . . 9
5.1.2 Function Documentation . . . 9
5.1.2.1 set_to_string() . . . 9
5.1.2.2 to_percentage_string() . . . 10
5.1.2.3 to_string() . . . 10
5.1.2.4 vector_to_string() . . . 11
5.2 core Namespace Reference . . . 11
5.2.1 Detailed Description . . . 12
5.2.2 Typedef Documentation . . . 12
5.2.2.1 Devices . . . 12
5.2.2.2 Servers . . . 12
5.3 data Namespace Reference . . . 12
5.4 data::generate Namespace Reference . . . 12
5.4.1 Detailed Description . . . 13
5.4.2 Function Documentation . . . 13
5.4.2.1 devices() . . . 13
5.4.2.2 servers() . . . 13
5.4.2.3 services() . . . 14
5.4.3 Variable Documentation . . . 14
5.4.3.1 DATA_PATH . . . 14
5.5 data::load Namespace Reference . . . 14
5.5.1 Detailed Description . . . 14
5.5.2 Function Documentation . . . 14
5.5.2.1 devices() . . . 14
5.5.2.2 servers() . . . 15
5.6 display Namespace Reference . . . 15
5.6.1 Detailed Description . . . 16
5.6.2 Function Documentation . . . 16
5.6.2.1 show_device() . . . 16
5.6.2.2 show_devices() . . . 16

5.6.2.3 show_metrics() . . . 16
5.6.2.4 show_metrics_console_or_file() . . . 16
5.6.2.5 show_server() . . . 17
5.6.2.6 show_servers() . . . 17
5.6.2.7 show_servers_usage() . . . 17
5.7 display::details Namespace Reference . . . 17
5.7.1 Detailed Description . . . 18
5.7.2 Function Documentation . . . 18
5.7.2.1 print_header() . . . 18
5.7.2.2 print_middle() . . . 18
5.7.2.3 print_row() . . . 19
5.7.2.4 print_title() . . . 19
5.7.3 Variable Documentation . . . 19
5.7.3.1 label_width . . . 19
5.7.3.2 total_width . . . 19
5.8 exact Namespace Reference . . . 19
5.8.1 Detailed Description . . . 20
5.8.2 Typedef Documentation . . . 20
5.8.2.1 NumVar2D . . . 20
5.8.3 Function Documentation . . . 20
5.8.3.1 boot() . . . 20
5.9 exact::anonymous_namespace{exact.hpp} Namespace Reference . . . 21
5.9.1 Function Documentation . . . 21
5.9.1.1 solver_maximum_cost() . . . 21
5.10 file_manager Namespace Reference . . . 21
5.11 file_manager::csv Namespace Reference . . . 21
5.11.1 Detailed Description . . . 22
5.11.2 Function Documentation . . . 22
5.11.2.1 append() . . . 22
5.11.2.2 count_lines() . . . 22
5.11.2.3 read() . . . 22
5.11.2.4 write() . . . 23
5.12 heuristic Namespace Reference . . . 23
5.12.1 Detailed Description . . . 23
5.12.2 Function Documentation . . . 23
5.12.2.1 boot() . . . 23
5.13 heuristic::anonymous_namespace{heuristic.hpp} Namespace Reference . . . 24
5.13.1 Function Documentation . . . 24
5.13.1.1 greedy_cost_benefit() . . . 24
5.13.1.2 greedy_random() . . . 24
5.13.1.3 greedy_random_fallback() . . . 25
5.14 metaheuristic Namespace Reference . . . 25

5.14.1 Detailed Description . . . 25
5.14.2 Function Documentation . . . 25
5.14.2.1 boot() . . . 25
5.15 metaheuristic::anonymous_namespace{metaheuristic.hpp} Namespace Reference . . . 26
5.15.1 Function Documentation . . . 26
5.15.1.1 apply_neighbor() . . . 26
5.15.1.2 revert_neighbor() . . . 27
5.15.1.3 simulated_annealing() . . . 27
5.15.2 Variable Documentation . . . 27
5.15.2.1 LOOP . . . 27
5.16 metrics Namespace Reference . . . 27
5.16.1 Detailed Description . . . 28
5.16.2 Enumeration Type Documentation . . . 28
5.16.2.1 Algorithm . . . 28
5.16.2.2 Paradigm . . . 29
5.16.3 Function Documentation . . . 29
5.16.3.1 calculate_metrics() . . . 29
5.16.3.2 save_results() . . . 29
5.16.3.3 to_string() [1/2] . . . 30
5.16.3.4 to_string() [2/2] . . . 30
5.16.4 Variable Documentation . . . 30
5.16.4.1 RESULTS_PATH . . . 30
5.17 randomness Namespace Reference . . . 30
5.17.1 Detailed Description . . . 31
5.17.2 Function Documentation . . . 31
5.17.2.1 get_engine() . . . 31
5.17.2.2 random_number() [1/2] . . . 31
5.17.2.3 random_number() [2/2] . . . 32
5.17.2.4 shuffled_range() . . . 32
5.18 sorting Namespace Reference . . . 33
5.18.1 Detailed Description . . . 33
5.18.2 Function Documentation . . . 34
5.18.2.1 sort_entities() [1/2] . . . 34
5.18.2.2 sort_entities() [2/2] . . . 34
5.19 sorting::details Namespace Reference . . . 35
5.19.1 Detailed Description . . . 35
5.19.2 Function Documentation . . . 35
5.19.2.1 sort_indices_impl() . . . 35

6 Class Documentation **37**

6.1 core::Device Struct Reference . . . 37
6.1.1 Detailed Description . . . 38

6.1.2 Constructor & Destructor Documentation . . . 38
6.1.2.1 Device() [1/2] . . . 38
6.1.2.2 Device() [2/2] . . . 38
6.1.3 Member Data Documentation . . . 38
6.1.3.1 cost . . . 38
6.1.3.2 cpu . . . 39
6.1.3.3 id . . . 39
6.1.3.4 mem . . . 39
6.1.3.5 served . . . 39
6.1.3.6 server_id . . . 39
6.1.3.7 service . . . 39
6.1.3.8 sto . . . 39
6.1.3.9 tasks . . . 40
6.2 metrics::ExactMetrics Struct Reference . . . 40
6.2.1 Detailed Description . . . 42
6.2.2 Constructor & Destructor Documentation . . . 42
6.2.2.1 ExactMetrics() [1/3] . . . 42
6.2.2.2 ExactMetrics() [2/3] . . . 42
6.2.2.3 ExactMetrics() [3/3] . . . 43
6.2.3 Member Function Documentation . . . 43
6.2.3.1 build_metrics() . . . 43
6.2.3.2 clone() . . . 43
6.2.3.3 data() . . . 44
6.2.3.4 get_header() . . . 44
6.2.4 Member Data Documentation . . . 44
6.2.4.1 gap . . . 44
6.2.4.2 objective_function . . . 44
6.2.4.3 status . . . 44
6.3 metrics::HeuristicMetrics Struct Reference . . . 45
6.3.1 Detailed Description . . . 46
6.3.2 Constructor & Destructor Documentation . . . 46
6.3.2.1 HeuristicMetrics() [1/3] . . . 46
6.3.2.2 HeuristicMetrics() [2/3] . . . 47
6.3.2.3 HeuristicMetrics() [3/3] . . . 47
6.3.3 Member Function Documentation . . . 47
6.3.3.1 clone() . . . 47
6.4 metrics::Metrics::Inputs Struct Reference . . . 48
6.4.1 Detailed Description . . . 48
6.4.2 Member Function Documentation . . . 48
6.4.2.1 operator=() . . . 48
6.4.3 Member Data Documentation . . . 49
6.4.3.1 devices_count . . . 49

6.4.3.2 max_penalty_cost . . . 49
6.4.3.3 servers_cc_count . . . 49
6.4.3.4 servers_ec_count . . . 49
6.5 metrics::MetaHeuristicMetrics Struct Reference . . . 49
6.5.1 Detailed Description . . . 51
6.5.2 Constructor & Destructor Documentation . . . 51
6.5.2.1 MetaHeuristicMetrics() [1/3] . . . 51
6.5.2.2 MetaHeuristicMetrics() [2/3] . . . 52
6.5.2.3 MetaHeuristicMetrics() [3/3] . . . 52
6.5.3 Member Function Documentation . . . 53
6.5.3.1 build_metrics() . . . 53
6.5.3.2 clone() . . . 53
6.5.3.3 data() . . . 53
6.5.3.4 get_header() . . . 53
6.5.4 Member Data Documentation . . . 54
6.5.4.1 alpha . . . 54
6.5.4.2 heuristic . . . 54
6.5.4.3 temperature . . . 54
6.6 metrics::MetricEntry Struct Reference . . . 54
6.6.1 Detailed Description . . . 54
6.6.2 Member Data Documentation . . . 55
6.6.2.1 category . . . 55
6.6.2.2 console_value . . . 55
6.6.2.3 file_value . . . 55
6.6.2.4 header . . . 55
6.6.2.5 label . . . 55
6.7 metrics::Metrics Struct Reference . . . 56
6.7.1 Detailed Description . . . 57
6.7.2 Constructor & Destructor Documentation . . . 57
6.7.2.1 Metrics() [1/4] . . . 57
6.7.2.2 Metrics() [2/4] . . . 58
6.7.2.3 Metrics() [3/4] . . . 58
6.7.2.4 Metrics() [4/4] . . . 58
6.7.2.5 ~Metrics() . . . 59
6.7.3 Member Function Documentation . . . 59
6.7.3.1 build_metrics() . . . 59
6.7.3.2 clone() . . . 59
6.7.3.3 data() . . . 59
6.7.3.4 get_base_directory_path() . . . 60
6.7.3.5 get_header() . . . 60
6.7.3.6 save_to_file() . . . 60
6.7.4 Member Data Documentation . . . 60

6.7.4.1 algorithm . . . 60
6.7.4.2 inputs . . . 60
6.7.4.3 outputs . . . 60
6.7.4.4 paradigm . . . 61
6.7.4.5 run_id . . . 61
6.8 metaheuristic::anonymous_namespace{metaheuristic.hpp}::Move Struct Reference . . . 61
6.8.1 Detailed Description . . . 61
6.8.2 Member Data Documentation . . . 62
6.8.2.1 delta_cost . . . 62
6.8.2.2 device_id . . . 62
6.8.2.3 new_server_id . . . 62
6.8.2.4 old_server_id . . . 62
6.8.2.5 swapped_id . . . 62
6.8.2.6 valid . . . 62
6.9 metrics::Metrics::Outputs Struct Reference . . . 62
6.9.1 Detailed Description . . . 63
6.9.2 Member Function Documentation . . . 63
6.9.2.1 operator=() . . . 63
6.9.3 Member Data Documentation . . . 63
6.9.3.1 devices_served_cc_count . . . 63
6.9.3.2 devices_served_count . . . 63
6.9.3.3 devices_served_ec_count . . . 64
6.9.3.4 execution_time . . . 64
6.9.3.5 servers_used_cc_count . . . 64
6.9.3.6 servers_used_count . . . 64
6.9.3.7 servers_used_ec_count . . . 64
6.9.3.8 total_penalty_cost . . . 64
6.10 metrics::Result Struct Reference . . . 64
6.10.1 Detailed Description . . . 65
6.10.2 Constructor & Destructor Documentation . . . 65
6.10.2.1 Result() [1/3] . . . 65
6.10.2.2 Result() [2/3] . . . 65
6.10.2.3 Result() [3/3] . . . 66
6.10.3 Member Function Documentation . . . 66
6.10.3.1 operator=() . . . 66
6.10.4 Member Data Documentation . . . 66
6.10.4.1 devices . . . 66
6.10.4.2 servers . . . 67
6.10.4.3 stats . . . 67
6.11 core::Server Struct Reference . . . 67
6.11.1 Detailed Description . . . 68
6.11.2 Constructor & Destructor Documentation . . . 68

6.11.2.1 Server() [1/2] . . . 68
6.11.2.2 Server() [2/2] . . . 68
6.11.3 Member Function Documentation . . . 69
6.11.3.1 add_service() . . . 69
6.11.3.2 can_serve() . . . 69
6.11.3.3 rmv_service() . . . 69
6.11.4 Member Data Documentation . . . 70
6.11.4.1 cpu . . . 70
6.11.4.2 id . . . 70
6.11.4.3 mem . . . 70
6.11.4.4 sto . . . 70
6.11.4.5 supply . . . 70
6.11.4.6 type . . . 70
6.12 core::Supply Struct Reference . . . 70
6.12.1 Detailed Description . . . 71
6.12.2 Constructor & Destructor Documentation . . . 71
6.12.2.1 Supply() [1/2] . . . 71
6.12.2.2 Supply() [2/2] . . . 71
6.12.3 Member Data Documentation . . . 72
6.12.3.1 cost . . . 72
6.12.3.2 cpu . . . 72
6.12.3.3 devices . . . 72
6.12.3.4 mem . . . 72
6.12.3.5 sto . . . 72

7 File Documentation **73**

7.1 include/exact.hpp File Reference . . . 73
7.1.1 Detailed Description . . . 74
7.2 exact.hpp . . . 74
7.3 include/heuristic.hpp File Reference . . . 76
7.3.1 Detailed Description . . . 77
7.4 heuristic.hpp . . . 77
7.5 include/metaheuristic.hpp File Reference . . . 79
7.5.1 Detailed Description . . . 80
7.6 metaheuristic.hpp . . . 81
7.7 include/mmkp.hpp File Reference . . . 83
7.7.1 Detailed Description . . . 87
7.7.2 Typedef Documentation . . . 87
7.7.2.1 i_matrix . . . 87
7.7.2.2 i_set . . . 87
7.7.2.3 i_vector . . . 87
7.7.2.4 id_pair . . . 87

7.7.2.5 id_pair_vector . . . 87
7.7.2.6 ii_pair . . . 87
7.7.2.7 ii_pair_vector . . . 88
7.7.2.8 s_matrix . . . 88
7.7.2.9 s_vector . . . 88
7.8 mmkp.hpp . . . 88
7.9 include/tools.hpp File Reference . . . 98
7.9.1 Detailed Description . . . 100
7.9.2 Variable Documentation . . . 101
7.9.2.1 DECIMAL_PLACES . . . 101
7.9.2.2 ZERO . . . 101
7.10 tools.hpp . . . 101
7.11 src/main.cpp File Reference . . . 104
7.11.1 Detailed Description . . . 105
7.11.2 Function Documentation . . . 106
7.11.2.1 main() . . . 106
7.11.2.2 run_simulation() . . . 106
7.11.3 Variable Documentation . . . 107
7.11.3.1 ALPHA . . . 107
7.11.3.2 CC_LENGTH . . . 107
7.11.3.3 CPLEX_TIME_LIMIT . . . 107
7.11.3.4 DEVICES_END . . . 107
7.11.3.5 DEVICES_START . . . 107
7.11.3.6 DEVICES_STEP . . . 107
7.11.3.7 EC_LENGTH . . . 107
7.11.3.8 MAX_RUNS . . . 108
7.11.3.9 SERVICES . . . 108
7.11.3.10 TEMPERATURE . . . 108

Index **109**

Chapter 1

Namespace Index

1.1 Namespace List

Here is a list of all namespaces with brief descriptions:

convert
: Short namespace alias for `file_manager::csv` . . . 9

core
: Contains the fundamental domain entities for the MMKP resource allocation problem . . . 11

data . . . 12

data::generate
: Generates synthetic datasets for services, IoT devices, and EC/CC servers when files are absent 12

data::load
: Parses raw string datasets into strongly-typed `core::Devices` and `core::Servers` vectors . . . 14

display
: Utility namespace for formatting and printing entities and ASCII metric tables to console or files 15

display::details
: Internal helper routines for rendering ASCII box-drawing tables . . . 17

exact
: Contains mathematical programming models for exact optimization . . . 19

exact::anonymous_namespace{exact.hpp} . . . 21

file_manager . . . 21

file_manager::csv
: Provides file I/O operations for reading, writing, and appending delimited CSV/text files . . . 21

heuristic
: Contains constructive heuristic algorithms for network resource allocation . . . 23

heuristic::anonymous_namespace{heuristic.hpp} . . . 24

metaheuristic
: Encapsulates meta-heuristic algorithms for network resource allocation . . . 25

metaheuristic::anonymous_namespace{metaheuristic.hpp} . . . 26

metrics
: Provides structures and routines for tracking, formatting, and persisting simulation metrics . . . 27

randomness
: Mersenne Twister (`std::mt19937`) random number generation and permutation utilities . . . 30

sorting
: Generic index-based sorting utilities for ordering 1-indexed entity vectors by member attributes 33

sorting::details
: Internal implementation helpers for pointer-to-member index sorting . . . 35

Chapter 2

Hierarchical Index

2.1 Class Hierarchy

This inheritance list is sorted roughly, but not completely, alphabetically:

core::Device 37
metrics::Metrics::Inputs 48
metrics::MetricEntry 54
metrics::Metrics 56
  metrics::ExactMetrics 40
  metrics::HeuristicMetrics 45
  metrics::MetaHeuristicMetrics 49
metaheuristic::anonymous_namespace{metaheuristic.hpp}::Move 61
metrics::Metrics::Outputs 62
metrics::Result 64
core::Server 67
core::Supply 70

Chapter 3

Class Index

3.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

core::Device
: Represents an IoT device requesting computational resources for a specific service 37

metrics::ExactMetrics
: Extends `Metrics` to include CPLEX solver status, objective value, and MIP optimality gap . . 40

metrics::HeuristicMetrics
: Specialization of `Metrics` for constructive heuristic runs 45

metrics::Metrics::Inputs
: Captures the static input instance dimensions and maximum possible penalty cost 48

metrics::MetaHeuristicMetrics
: Extends `Metrics` to track Simulated Annealing hyperparameters and initial heuristic selection 49

metrics::MetricEntry
: Represents a formatted key-value entry for displaying or exporting simulation metrics 54

metrics::Metrics
: Polymorphic base structure for tracking, calculating, and saving simulation metrics 56

metaheuristic::anonymous_namespace{metaheuristic.hpp}::Move
: Represents a reversible state transition within the neighborhood search 61

metrics::Metrics::Outputs
: Captures the resulting performance and resource utilization metrics after allocation 62

metrics::Result
: Encapsulates the complete system state (`Devices`, `Servers`) and polymorphic `Metrics` for a simulation run 64

core::Server
: Represents an Edge Computing (EC) or Cloud Computing (CC) server node 67

core::Supply
: Tracks the cumulative resource consumption and allocated device IDs on a single server . . . 70

Chapter 4

File Index

4.1 File List

Here is a list of all files with brief descriptions:

include/exact.hpp
Exact Integer Linear Programming (ILP) formulations using IBM ILOG CPLEX 73
include/heuristic.hpp
Constructive heuristic algorithms for fast MMKP resource allocation 76
include/metaheuristic.hpp
Meta-heuristic optimization algorithms and neighborhood operators for MMKP 79
include/mmkp.hpp
Core data structures, synthetic dataset generators, metric trackers, and console/file display routines 83
include/tools.hpp
General-purpose utilities for CSV I/O, string formatting, random number generation, and indirect sorting 98
src/main.cpp
Entry point and benchmark runner for the MMKP IoT-EC-CC resource allocation simulator . . . 104

Chapter 5

Namespace Documentation

5.1 convert Namespace Reference

Short namespace alias for `file_manager::csv`.

Functions

- template<typename T >
  std::string to_string (const T &value, int precision=DECIMAL_PLACES)
  Converts a value of any streamable type to its string representation.
- std::string to_percentage_string (double numerator, double denominator, int precision=DECIMAL_PLACES)
  Calculates a percentage and converts it to a formatted string.
- template<typename T >
  std::string vector_to_string (const std::vector< T > &vec)
  Converts a vector of primitive types to a bracketed array string representation.
- template<typename T >
  std::string set_to_string (const std::set< T > &s)
  Converts a `std::set` of primitive types to a bracketed array string representation.

5.1.1 Detailed Description

Short namespace alias for `file_manager::csv`.

String conversion and formatting utilities for numbers, percentages, vectors, and sets.

5.1.2 Function Documentation

5.1.2.1 set_to_string()

```
template<typename T >
std::string convert::set_to_string (
            const std::set< T > & s )  [inline]
```

Converts a `std::set` of primitive types to a bracketed array string representation.

Template Parameters

T	The type of elements in the set.

Parameters

s	The set to convert.

Returns

A string formatted as "`[e1,e2,...]`".

5.1.2.2 to_percentage_string()

```
std::string convert::to_percentage_string (
            double numerator,
            double denominator,
            int precision = DECIMAL_PLACES )  [inline]
```

Calculates a percentage and converts it to a formatted string.

Computes $(\text{numerator} / \text{denominator}) \times 100.0$ and formats it to `precision` decimal places. Safely returns `0.0` when `denominator == 0.0`.

Parameters

numerator	The value representing the part of the total.
denominator	The value representing the total.
precision	The number of decimal places for the resulting percentage string.

Returns

A string representation of the calculated percentage.

5.1.2.3 to_string()

```
template<typename T >
std::string convert::to_string (
            const T & value,
            int precision = DECIMAL_PLACES )  [inline]
```

Converts a value of any streamable type to its string representation.

Uses `std::ostringstream` for conversion, applying fixed-point notation with the specified precision for floating-point types.

Template Parameters

T	The data type of the value to be converted.

Parameters

value	The value to convert.
precision	The number of decimal places for floating-point types.

Returns

A formatted string representation of `value`.

5.1.2.4 vector_to_string()

```
template<typename T >
std::string convert::vector_to_string (
            const std::vector< T > & vec )  [inline]
```

Converts a vector of primitive types to a bracketed array string representation.

Template Parameters

T	The type of elements in the vector.

Parameters

vec	The vector to convert.

Returns

A string formatted as `"[e1,e2,...]"`.

5.2 core Namespace Reference

Contains the fundamental domain entities for the MMKP resource allocation problem.

Classes

- struct Device

  Represents an IoT device requesting computational resources for a specific service.
- struct Server

  Represents an Edge Computing (EC) or Cloud Computing (CC) server node.
- struct Supply

  Tracks the cumulative resource consumption and allocated device IDs on a single server.

Typedefs

- using Devices = std::vector< Device >

  Contiguous collection of `Device` instances (1-indexed, index 0 is a placeholder).
- using Servers = std::vector< Server >

  Contiguous collection of `Server` instances (1-indexed, index 0 is a placeholder).

5.2.1 Detailed Description

Contains the fundamental domain entities for the MMKP resource allocation problem.

5.2.2 Typedef Documentation

5.2.2.1 Devices

```
using core::Devices = typedef std::vector<Device>
```

Contiguous collection of `Device` instances (1-indexed, index 0 is a placeholder).

5.2.2.2 Servers

```
using core::Servers = typedef std::vector<Server>
```

Contiguous collection of `Server` instances (1-indexed, index 0 is a placeholder).

5.3 data Namespace Reference

Namespaces

- namespace generate

  Generates synthetic datasets for services, IoT devices, and EC/CC servers when files are absent.
- namespace load

  Parses raw string datasets into strongly-typed `core::Devices` and `core::Servers` vectors.

5.4 data::generate Namespace Reference

Generates synthetic datasets for services, IoT devices, and EC/CC servers when files are absent.

Functions

- std::vector< std::vector< std::string > > services (const int num_services=10)

  Generates or loads network service requirement templates.
- std::vector< std::vector< std::string > > devices (const int length, const int num_services=10)

  Generates or loads device request data representing IoT network demands.
- std::vector< std::vector< std::string > > servers (const int ec_length=99, const int cc_length=1)

  Generates or loads server infrastructure data for Edge and Cloud nodes.

Variables

- const std::filesystem::path DATA_PATH = std::filesystem::current_path() / "data"

  Path pointing to the root `data/` directory.

5.4.1 Detailed Description

Generates synthetic datasets for services, IoT devices, and EC/CC servers when files are absent.

5.4.2 Function Documentation

5.4.2.1 devices()

```
std::vector< std::vector< std::string > > data::generate::devices (
            const int length,
            const int num_services = 10 )  [inline]
```

Generates or loads device request data representing IoT network demands.

Uniformly samples service requirement profiles from the services dataset and assigns them to each device instance. Saves the output to `data/devices/Devices_<length>.txt`.

Parameters

length	The total number of devices to generate.
num_services	The number of underlying service templates to sample from.

Returns

A 2D string vector representing the devices dataset.

5.4.2.2 servers()

```
std::vector< std::vector< std::string > > data::generate::servers (
            const int ec_length = 99,
            const int cc_length = 1 )  [inline]
```

Generates or loads server infrastructure data for Edge and Cloud nodes.

Creates capacity profiles for `ec_length` Edge servers ('E') and cc_length`Cloud servers ('C'), saving the table to` data/servers/servers_<ec>_<cc>.txt`.

Parameters

ec_length	The number of Edge Computing servers to generate.
cc_length	The number of Cloud Computing servers to generate.

Returns

A 2D string vector representing the servers dataset.

5.4.2.3 services()

```
std::vector< std::vector< std::string > > data::generate::services (
            const int num_services = 10 )  [inline]
```

Generates or loads network service requirement templates.

Creates random service profiles with varying task counts, memory, storage, and processing demands. Persists the resulting table to `data/Services.txt` for reproducibility.

Parameters

num_services	The total number of distinct service profiles to generate.

Returns

A 2D string vector containing the generated or loaded services dataset.

5.4.3 Variable Documentation

5.4.3.1 DATA_PATH

```
const std::filesystem::path data::generate::DATA_PATH = std::filesystem::current_path() /
"data"
```

Path pointing to the root `data/` directory.

5.5 data::load Namespace Reference

Parses raw string datasets into strongly-typed `core::Devices` and `core::Servers` vectors.

Functions

- core::Devices devices (const int length, const int num_services=10)

  Loads device data from disk (generating it if absent) into a vector of `core::Device` objects.
- core::Servers servers (const int ec=99, const int cc=1)

  Loads Edge and Cloud server data from disk (generating it if absent).

5.5.1 Detailed Description

Parses raw string datasets into strongly-typed `core::Devices` and `core::Servers` vectors.

5.5.2 Function Documentation

5.5.2.1 devices()

```
core::Devices data::load::devices (
            const int length,
            const int num_services = 10 )  [inline]
```

Loads device data from disk (generating it if absent) into a vector of `core::Device` objects.

Parameters

length	The total number of devices to load.
num_services	The number of distinct services to generate if absent. Defaults to 10.

Returns

A 1-indexed `core::Devices` vector containing the parsed device instances.

5.5.2.2 servers()

```
core::Servers data::load::servers (
            const int ec = 99,
            const int cc = 1 )  [inline]
```

Loads Edge and Cloud server data from disk (generating it if absent).

Parameters

ec	The number of Edge Computing (EC) servers to load.
cc	The number of Cloud Computing (CC) servers to load.

Returns

A 1-indexed `core::Servers` vector containing both EC and CC server instances.

5.6 display Namespace Reference

Utility namespace for formatting and printing entities and ASCII metric tables to console or files.

Namespaces

- namespace details

  Internal helper routines for rendering ASCII box-drawing tables.

Functions

- void show_device (const core::Device &device)

  Displays detailed information of a single device to standard output.
- void show_devices (const core::Devices &devices)

  Displays detailed information for all devices in the vector.
- void show_server (const core::Server &server)

  Displays capacity and current demand of a single server.
- void show_servers (const core::Servers &servers)

  Displays capacity and demand information for all servers.
- void show_metrics_console_or_file (const metrics::Metrics &metrics, std::ostream &out)

  Formats and writes the complete simulation metrics table to an output stream.
- void show_metrics (const std::shared_ptr< metrics::Metrics > &ptr)

  Displays the metrics table on the console and appends it to a drawing file.
- void show_servers_usage (const core::Servers &servers)

  Prints a summary table of server resource utilization percentages.

5.6.1 Detailed Description

Utility namespace for formatting and printing entities and ASCII metric tables to console or files.

5.6.2 Function Documentation

5.6.2.1 show_device()

```
void display::show_device (
            const core::Device & device )  [inline]
```

Displays detailed information of a single device to standard output.

Parameters

device	The device instance to display.

5.6.2.2 show_devices()

```
void display::show_devices (
            const core::Devices & devices )  [inline]
```

Displays detailed information for all devices in the vector.

Parameters

devices	The vector of devices to display.

5.6.2.3 show_metrics()

```
void display::show_metrics (
            const std::shared_ptr< metrics::Metrics > & ptr )  [inline]
```

Displays the metrics table on the console and appends it to a drawing file.

Parameters

ptr	Shared pointer to the simulation metrics instance.

5.6.2.4 show_metrics_console_or_file()

```
void display::show_metrics_console_or_file (
            const metrics::Metrics & metrics,
            std::ostream & out )  [inline]
```

Formats and writes the complete simulation metrics table to an output stream.

Parameters

metrics	The metrics instance to format.
out	The target output stream.

5.6.2.5 show_server()

```
void display::show_server (
            const core::Server & server )  [inline]
```

Displays capacity and current demand of a single server.

Parameters

server	The server instance to display.

5.6.2.6 show_servers()

```
void display::show_servers (
            const core::Servers & servers )  [inline]
```

Displays capacity and demand information for all servers.

Parameters

servers	The vector of servers to display.

5.6.2.7 show_servers_usage()

```
void display::show_servers_usage (
            const core::Servers & servers )  [inline]
```

Prints a summary table of server resource utilization percentages.

Parameters

servers	The vector of servers to summarize.

5.7 display::details Namespace Reference

Internal helper routines for rendering ASCII box-drawing tables.

Functions

- void print_row (std::ostream &out, const std::string &label, const std::string &value)

  Prints a formatted key-value row within the ASCII table.
- void print_header (std::ostream &out)

  Prints the top or bottom border of the ASCII table.
- void print_middle (std::ostream &out)

  Prints a section separator line within the ASCII table.
- void print_title (std::ostream &out, const std::string &title)

  Prints a centered title row within the ASCII table.

Variables

- const int total_width = 72

  Total character width of the formatted ASCII table.
- const int label_width = 26

  Character width allocated to the left-hand label column.

5.7.1 Detailed Description

Internal helper routines for rendering ASCII box-drawing tables.

5.7.2 Function Documentation

5.7.2.1 print_header()

```
void display::details::print_header (
            std::ostream & out )  [inline]
```

Prints the top or bottom border of the ASCII table.

Parameters

out	The target output stream.

5.7.2.2 print_middle()

```
void display::details::print_middle (
            std::ostream & out )  [inline]
```

Prints a section separator line within the ASCII table.

Parameters

out	The target output stream.

5.7.2.3 print_row()

```
void display::details::print_row (
            std::ostream & out,
            const std::string & label,
            const std::string & value )  [inline]
```

Prints a formatted key-value row within the ASCII table.

Parameters

out	The target output stream.
label	The row label string.
value	The row value string.

5.7.2.4 print_title()

```
void display::details::print_title (
            std::ostream & out,
            const std::string & title )  [inline]
```

Prints a centered title row within the ASCII table.

Parameters

out	The target output stream.
title	The title text to center.

5.7.3 Variable Documentation

5.7.3.1 label_width

```
const int display::details::label_width = 26
```

Character width allocated to the left-hand label column.

5.7.3.2 total_width

```
const int display::details::total_width = 72
```

Total character width of the formatted ASCII table.

5.8 exact Namespace Reference

Contains mathematical programming models for exact optimization.

Namespaces

- namespace anonymous_namespace{exact.hpp}

Typedefs

- using NumVar2D = IloArray< IloNumVarArray >

  Type alias for a 2D matrix of CPLEX decision variables.

Functions

- void boot (metrics::Result &state, int time_limit_sec=3600)

  Bootstraps and executes the configured exact mathematical model.

5.8.1 Detailed Description

Contains mathematical programming models for exact optimization.

Utilizes the IBM ILOG CPLEX optimizer to solve the network resource allocation problem to optimality or within a specified time limit, providing an exact baseline by maximizing avoided non-service penalties.

5.8.2 Typedef Documentation

5.8.2.1 NumVar2D

```
using exact::NumVar2D = typedef IloArray<IloNumVarArray>
```

Type alias for a 2D matrix of CPLEX decision variables.

5.8.3 Function Documentation

5.8.3.1 boot()

```
void exact::boot (
            metrics::Result & state,
            int time_limit_sec = 3600 )  [inline]
```

Bootstraps and executes the configured exact mathematical model.

Reads the specified algorithm identifier from `state.stats` and dispatches execution to the corresponding exact solver implementation.

Parameters

state	The unified simulation state to be processed.
time_limit_sec	The maximum execution time allowed for the exact solver in seconds.

5.9 exact::anonymous_namespace{exact.hpp} Namespace Reference

Functions

- void solver_maximum_cost (metrics::Result &state, int time_limit_sec=3600)

5.9.1 Function Documentation

5.9.1.1 solver_maximum_cost()

```
void exact::anonymous_namespace{exact.hpp}::solver_maximum_cost (
            metrics::Result & state,
            int time_limit_sec = 3600 )  [inline]
```

$ indicating whether device d is assigned to server i. Maximizes the total utility of served devices (avoided SLA penalties) subject to single-server assignment and multidimensional resource capacity constraints (CPU, MEM, STO).

Parameters

state	The unified simulation state containing devices, servers, and metrics.
time_limit_sec	The maximum execution time allowed for the solver in seconds.

5.10 file_manager Namespace Reference

Namespaces

- namespace csv

  Provides file I/O operations for reading, writing, and appending delimited CSV/text files.

5.11 file_manager::csv Namespace Reference

Provides file I/O operations for reading, writing, and appending delimited CSV/text files.

Functions

- std::optional< std::vector< std::vector< std::string > > > read (const std::string &file_path, char delimiter=',')

  Reads a delimited CSV file into a 2D vector of strings.
- bool write (const std::string &file_path, const std::vector< std::vector< std::string > > &data, char delimiter=',')

  Writes a 2D vector of strings to a CSV file, creating parent directories if needed.
- bool append (const std::string &file_path, const std::vector< std::vector< std::string > > &data, char delimiter=',')

  Appends a 2D vector of strings to an existing or new CSV file.
- int count_lines (const std::string &file_path)

  Counts the total number of lines in a file.

5.11.1 Detailed Description

Provides file I/O operations for reading, writing, and appending delimited CSV/text files.

5.11.2 Function Documentation

5.11.2.1 append()

```
bool file_manager::csv::append (
            const std::string & file_path,
            const std::vector< std::vector< std::string > > & data,
            char delimiter = ',' )  [inline]
```

Appends a 2D vector of strings to an existing or new CSV file.

Parameters

file_path	The full path to the output file.
data	The 2D vector of strings to append.
delimiter	The character used for separating columns (defaults to ‘,’) . @returntrueif the data was appended successfully;false` otherwise.

5.11.2.2 count_lines()

```
int file_manager::csv::count_lines (
            const std::string & file_path )  [inline]
```

Counts the total number of lines in a file.

Parameters

file_path	The full path to the file.

Returns

The total number of lines, or 0 if the file does not exist or cannot be opened.

5.11.2.3 read()

```
std::optional< std::vector< std::vector< std::string > > > file_manager::csv::read (
            const std::string & file_path,
            char delimiter = ',' )  [inline]
```

Reads a delimited CSV file into a 2D vector of strings.

Parameters

file_path	The full path to the file to be read.
delimiter	The character that separates columns (defaults to ‘,’) . @return Anstd::optionalcontaining the 2D vector of strings upon success, orstd::nullopt` on failure.

5.11.2.4 write()

```
bool file_manager::csv::write (
            const std::string & file_path,
            const std::vector< std::vector< std::string > > & data,
            char delimiter = ',' )  [inline]
```

Writes a 2D vector of strings to a CSV file, creating parent directories if needed.

Parameters

file_path	The full path to the output file.
data	The 2D vector of strings to write.
delimiter	The character used for separating columns (defaults to ‘,') . @returntrueif the file was written successfully;false` otherwise.

5.12 heuristic Namespace Reference

Contains constructive heuristic algorithms for network resource allocation.

Namespaces

- namespace anonymous_namespace{heuristic.hpp}

Functions

- void boot (metrics::Result &state)

  Bootstraps and executes the configured constructive heuristic.

5.12.1 Detailed Description

Contains constructive heuristic algorithms for network resource allocation.

Implements fast, polynomial-time approximation strategies (Greedy Cost-Benefit, Greedy Random, and Greedy Random Fallback) prioritizing low execution times while minimizing unserved device penalty costs.

5.12.2 Function Documentation

5.12.2.1 boot()

```
void heuristic::boot (
            metrics::Result & state )  [inline]
```

Bootstraps and executes the configured constructive heuristic.

Reads the specified algorithm identifier from `state.stats` and dispatches execution to the corresponding heuristic solver.

Parameters

state	The unified simulation state to be processed.

5.13 heuristic::anonymous_namespace{heuristic.hpp} Namespace Reference

Functions

- void greedy_cost_benefit (metrics::Result &state)

  Executes a Global Greedy Resource Allocation based on surrogate pseudo-utility.
- void greedy_random (metrics::Result &state)

  Executes a fast random baseline resource allocation.
- void greedy_random_fallback (metrics::Result &state)

  Executes a two-stage random fallback resource allocation.

5.13.1 Function Documentation

5.13.1.1 greedy_cost_benefit()

```
void heuristic::anonymous_namespace{heuristic.hpp}::greedy_cost_benefit (
            metrics::Result & state )  [inline]
```

Executes a Global Greedy Resource Allocation based on surrogate pseudo-utility.

Normalizes device resource demands against total system capacity to compute a multidimensional cost-benefit pseudo-utility and sorts candidates once globally. Allocates devices greedily per server using prefix-skipping and minimum-demand early termination.

Parameters

state	The unified simulation result state containing devices, servers, and metrics.

5.13.1.2 greedy_random()

```
void heuristic::anonymous_namespace{heuristic.hpp}::greedy_random (
            metrics::Result & state )  [inline]
```

Executes a fast random baseline resource allocation.

Iterates through all devices and attempts to assign each one to a single uniformly selected server. If the selected server lacks capacity, the device remains unserved. Metric calculation is deferred.

Parameters

state	The unified simulation result state containing devices, servers, and metrics.

5.13.1.3 greedy_random_fallback()

```
void heuristic::anonymous_namespace{heuristic.hpp}::greedy_random_fallback (
            metrics::Result & state )  [inline]
```

Executes a two-stage random fallback resource allocation.

Attempts to assign each device to a randomly selected Edge (EC) server first. If the selected Edge server lacks capacity, falls back to a randomly selected Cloud (CC) server.

Parameters

state	The unified simulation result state containing devices, servers, and metrics.

5.14 metaheuristic Namespace Reference

Encapsulates meta-heuristic algorithms for network resource allocation.

Namespaces

- namespace anonymous_namespace{metaheuristic.hpp}

Functions

- void boot (metrics::Result &state)

  Initializes and launches the selected meta-heuristic algorithm.

5.14.1 Detailed Description

Encapsulates meta-heuristic algorithms for network resource allocation.

Provides trajectory-based optimization techniques, specifically Simulated Annealing, to iteratively improve initial heuristic solutions by exploring shift, insertion, and 1-for-1 swap neighborhood structures.

5.14.2 Function Documentation

5.14.2.1 boot()

```
void metaheuristic::boot (
            metrics::Result & state )  [inline]
```

Initializes and launches the selected meta-heuristic algorithm.

Generates an initial solution using the configured constructive base heuristic, records its standalone metrics in the heuristic results directory, and then executes the meta-heuristic optimization over the resulting state.

Parameters

state	The unified simulation state to be optimized.

5.15 metaheuristic::anonymous_namespace{metaheuristic.hpp} Namespace Reference

Classes

- struct Move

  Represents a reversible state transition within the neighborhood search.

Functions

- Move apply_neighbor (core::Devices &devices, core::Servers &servers, int tries=10)

  Applies a shift, insert, or swap neighborhood move to the current state.
- void revert_neighbor (core::Devices &devices, core::Servers &servers, const Move &move)

  Reverts a previously applied neighborhood move in $O(\log N)$ time.
- void simulated_annealing (metrics::Result &state)

  Executes the Simulated Annealing (SA) optimization algorithm.

Variables

- constexpr int LOOP = 10

  Number of inner-loop iterations (Markov chain length) executed per temperature level.

5.15.1 Function Documentation

5.15.1.1 apply_neighbor()

```
Move metaheuristic::anonymous_namespace{metaheuristic.hpp}::apply_neighbor (
            core::Devices & devices,
            core::Servers & servers,
            int tries = 10 )  [inline]
```

Applies a shift, insert, or swap neighborhood move to the current state.

Selects a random device and attempts to allocate or shift it across random servers (including a fallback check on the last server for unserved devices). If direct allocation fails for an unserved device, attempts a 1-for-1 swap with a currently served device.

Parameters

devices	The global state vector of devices.
servers	The global state vector of servers.
tries	The maximum number of random candidate attempts.

Returns

Move A structure containing the applied transition details and cost variation.

5.15.1.2 revert_neighbor()

```
void metaheuristic::anonymous_namespace{metaheuristic.hpp}::revert_neighbor (
            core::Devices & devices,
            core::Servers & servers,
            const Move & move )  [inline]
```

Reverts a previously applied neighborhood move in $O(\log N)$ time.

Restores the exact server allocations and resource supplies prior to `apply_neighbor`.

Parameters

devices	The global state vector of devices.
servers	The global state vector of servers.
move	The `Move` structure containing the transition details to undo.

5.15.1.3 simulated_annealing()

```
void metaheuristic::anonymous_namespace{metaheuristic.hpp}::simulated_annealing (
            metrics::Result & state )  [inline]
```

Executes the Simulated Annealing (SA) optimization algorithm.

Iteratively explores the solution space by applying random neighborhood moves. Accepts worsening moves according to the Metropolis criterion $\exp(-\Delta / T)$ to escape local optima, cooling down geometrically via factor α until $T \le 10^{-3}$. Uses lazy synchronization for tracking the best-found state.

Parameters

state	The unified simulation state containing devices, servers, and metrics.

5.15.2 Variable Documentation

5.15.2.1 LOOP

```
constexpr int metaheuristic::anonymous_namespace{metaheuristic.hpp}::LOOP = 10  [constexpr]
```

Number of inner-loop iterations (Markov chain length) executed per temperature level.

5.16 metrics Namespace Reference

Provides structures and routines for tracking, formatting, and persisting simulation metrics.

Classes

- struct ExactMetrics

  Extends `Metrics` to include CPLEX solver status, objective value, and MIP optimality gap.
- struct HeuristicMetrics

  Specialization of `Metrics` for constructive heuristic runs.
- struct MetaHeuristicMetrics

  Extends `Metrics` to track Simulated Annealing hyperparameters and initial heuristic selection.
- struct MetricEntry

  Represents a formatted key-value entry for displaying or exporting simulation metrics.
- struct Metrics

  Polymorphic base structure for tracking, calculating, and saving simulation metrics.
- struct Result

  Encapsulates the complete system state (`Devices`, `Servers`) and polymorphic `Metrics` for a simulation run.

Enumerations

- enum class Paradigm { NONE , EXACT , HEURISTIC , METAHEURISTIC }

  Enumerates the optimization paradigms supported by the simulator.
- enum class Algorithm {
  NONE , solver_maximum_cost , greedy_cost_benefit , greedy_random ,
  greedy_random_fallback , simulated_annealing }

  Identifies the specific optimization algorithm used in a simulation run.

Functions

- std::string to_string (Paradigm paradigm)

  Converts a `Paradigm` enum value to its uppercase string representation.
- std::string to_string (Algorithm algorithm)

  Converts an `Algorithm` enum value to its standardized string identifier.
- void calculate_metrics (Result &state)

  Computes the final simulation metrics based on the current device-to-server allocations.
- void save_results (const Result &state, bool show_servers_usage=false)

  Saves the final calculated metrics and ASCII summary tables to the file system.

Variables

- const std::filesystem::path RESULTS_PATH = std::filesystem::current_path() / "results"

  Path pointing to the root `results/` directory.

5.16.1 Detailed Description

Provides structures and routines for tracking, formatting, and persisting simulation metrics.

5.16.2 Enumeration Type Documentation

5.16.2.1 Algorithm

```
enum class metrics::Algorithm  [strong]
```

Identifies the specific optimization algorithm used in a simulation run.

Enumerator

NONE	Uninitialized algorithm.
solver_maximum_cost	Exact ILP solver maximizing avoided penalty cost.
greedy_cost_benefit	Deterministic surrogate cost-benefit greedy heuristic.
greedy_random	Single-stage uniform random greedy heuristic.
greedy_random_fallback	Two-stage Edge-first random greedy heuristic with Cloud fallback.
simulated_annealing	Simulated Annealing meta-heuristic.

5.16.2.2 Paradigm

```
enum class metrics::Paradigm  [strong]
```

Enumerates the optimization paradigms supported by the simulator.

Enumerator

NONE	Uninitialized paradigm.
EXACT	Exact mathematical programming (ILP via CPLEX).
HEURISTIC	Constructive heuristic algorithms.
METAHEURISTIC	Meta-heuristic optimization algorithms.

5.16.3 Function Documentation

5.16.3.1 calculate_metrics()

```
void metrics::calculate_metrics (
            Result & state )  [inline]
```

Computes the final simulation metrics based on the current device-to-server allocations.

Parameters

state	The simulation result structure holding `devices`, `servers`, and `stats`.

5.16.3.2 save_results()

```
void metrics::save_results (
            const Result & state,
            bool show_servers_usage = false )  [inline]
```

Saves the final calculated metrics and ASCII summary tables to the file system.

Parameters

state	The simulation result structure to be saved.
show_servers_usage	Flag to indicate if individual server resource usage should be printed.

5.16.3.3 to_string() [1/2]

```
std::string metrics::to_string (
            Algorithm algorithm )  [inline]
```

Converts an `Algorithm` enum value to its standardized string identifier.

Parameters

algorithm	The `Algorithm` value to convert.

Returns

String representation of the algorithm.

5.16.3.4 to_string() [2/2]

```
std::string metrics::to_string (
            Paradigm paradigm )  [inline]
```

Converts a `Paradigm` enum value to its uppercase string representation.

Parameters

paradigm	The `Paradigm` value to convert.

Returns

String representation of the paradigm.

5.16.4 Variable Documentation

5.16.4.1 RESULTS_PATH

```
const std::filesystem::path metrics::RESULTS_PATH = std::filesystem::current_path() / "results"
```

Path pointing to the root `results/` directory.

5.17 randomness Namespace Reference

Mersenne Twister (`std::mt19937`) random number generation and permutation utilities.

Functions

- std::mt19937 & get_engine ()

  Provides a singleton instance of a pseudo-random number engine.
- template<typename T >
  T random_number (T min, T max)

  Generates a uniform random number within a specified inclusive interval $[\text{min}, \text{max}]$.
- template<typename T >
  T random_number (T min, T max, std::vector< T > &exclude)

  Generates a random number within $[\text{min}, \text{max}]$, excluding values present in `exclude`*.*
- template<typename T >
  std::vector< T > shuffled_range (T min, T max)

  Creates a randomly shuffled vector containing all integers in the inclusive range $[\text{min}, \text{max}]$.

5.17.1 Detailed Description

Mersenne Twister (`std::mt19937`) random number generation and permutation utilities.

5.17.2 Function Documentation

5.17.2.1 get_engine()

```
std::mt19937 & randomness::get_engine ( )  [inline]
```

Provides a singleton instance of a pseudo-random number engine.

Returns a reference to a static `std::mt19937` engine seeded once via `std::random_device`.

Returns

A reference to the static `std::mt19937` random engine.

5.17.2.2 random_number() [1/2]

```
template<typename T >
T randomness::random_number (
            T min,
            T max )  [inline]
```

Generates a uniform random number within a specified inclusive interval $[\text{min}, \text{max}]$.

Template Parameters

T	The numeric type (must satisfy `std::is_arithmetic_v<T>`).

Parameters

min	The lower bound of the interval (inclusive).

Parameters

max	The upper bound of the interval (inclusive).

Returns

A uniformly distributed random number within $[\text{min}, \text{max}]$.

Exceptions

std::invalid_argument	If `min > max`.

5.17.2.3 random_number() [2/2]

```
template<typename T >
T randomness::random_number (
            T min,
            T max,
            std::vector< T > & exclude )  [inline]
```

Generates a random number within $[\text{min}, \text{max}]$, excluding values present in `exclude`.

The generated number is automatically appended to `exclude` to prevent future duplicates.

Template Parameters

T	The numeric type (must satisfy `std::is_arithmetic_v<T>`).

Parameters

min	The lower bound of the interval (inclusive).
max	The upper bound of the interval (inclusive).
exclude	A reference to a vector containing values to be excluded.

Returns

A random number within $[\text{min}, \text{max}]$ not previously in `exclude`.

Exceptions

std::invalid_argument	If `min > max`, or if all possible integral values in the range are already excluded.

5.17.2.4 shuffled_range()

```
template<typename T >
std::vector< T > randomness::shuffled_range (
```

```
    T min,
    T max )  [inline]
```

Creates a randomly shuffled vector containing all integers in the inclusive range $[\text{min}, \text{max}]$.

Template Parameters

T	The integral type for the range.

Parameters

min	The lower bound of the range (inclusive).
max	The upper bound of the range (inclusive).

Returns

A randomly shuffled `std::vector<T>` of unique integers from `min` to `max`.

Exceptions

std::invalid_argument	If `min > max`.

5.18 sorting Namespace Reference

Generic index-based sorting utilities for ordering 1-indexed entity vectors by member attributes.

Namespaces

- namespace details

  Internal implementation helpers for pointer-to-member index sorting.

Functions

- template<bool Ascending, typename T , typename AttributeType , AttributeType T::* AttributePtr>
  std::vector< int > sort_entities (const std::vector< T > &entities)

  Sorts the indices of all entities (from `1` to `N-1`) by a specific struct/class attribute.
- template<bool Ascending, typename T , typename AttributeType , AttributeType T::* AttributePtr>
  std::vector< int > sort_entities (const std::vector< T > &entities, const std::vector< int > &indices_to_sort)

  Sorts a specific subset of entity indices by a struct/class member attribute.

5.18.1 Detailed Description

Generic index-based sorting utilities for ordering 1-indexed entity vectors by member attributes.

5.18.2 Function Documentation

5.18.2.1 sort_entities() [1/2]

```
template<bool Ascending, typename T , typename AttributeType , AttributeType T::* Attribute↩
Ptr>
std::vector< int > sorting::sort_entities (
            const std::vector< T > & entities )  [inline]
```

Sorts the indices of all entities (from `1` to `N-1`) by a specific struct/class attribute.

Does not mutate the original `entities` vector. Assumes 1-based indexing where index `0` is a placeholder.

Template Parameters

Ascending	If `true`, sorts in ascending order; otherwise, descending.
T	The type of the entity.
AttributeType	The data type of the member attribute.
AttributePtr	Pointer-to-member specifying the attribute for comparison.

Parameters

entities	The constant 1-indexed vector of entities.

Returns

A new `std::vector<int>` containing the sorted 1-based indices.

5.18.2.2 sort_entities() [2/2]

```
template<bool Ascending, typename T , typename AttributeType , AttributeType T::* Attribute↩
Ptr>
std::vector< int > sorting::sort_entities (
            const std::vector< T > & entities,
            const std::vector< int > & indices_to_sort )  [inline]
```

Sorts a specific subset of entity indices by a struct/class member attribute.

Template Parameters

Ascending	If `true`, sorts in ascending order; otherwise, descending.
T	The type of the entity.
AttributeType	The data type of the member attribute.
AttributePtr	Pointer-to-member specifying the attribute for comparison.

Parameters

entities	The constant 1-indexed vector of entities.
indices_to_sort	A vector of integers representing the subset of 1-based indices to sort.

Returns

A new `std::vector<int>` containing the sorted subset indices.

5.19 sorting::details Namespace Reference

Internal implementation helpers for pointer-to-member index sorting.

Functions

- template<bool Ascending, typename T , typename AttributeType , AttributeType T::* AttributePtr>
  void sort_indices_impl (const std::vector< T > &entities, std::vector< int > &idxs)

  Sorts a vector of indices based on a specified struct/class member attribute.

5.19.1 Detailed Description

Internal implementation helpers for pointer-to-member index sorting.

5.19.2 Function Documentation

5.19.2.1 sort_indices_impl()

```
template<bool Ascending, typename T , typename AttributeType , AttributeType T::* Attribute↩
Ptr>
void sorting::details::sort_indices_impl (
            const std::vector< T > & entities,
            std::vector< int > & idxs )
```

Sorts a vector of indices based on a specified struct/class member attribute.

Template Parameters

Ascending	Sort direction flag (`true` for ascending, `false` for descending).
T	Entity struct/class type.
AttributeType	Data type of the sorted member attribute.
AttributePtr	Pointer-to-member specifying the comparison attribute.

Parameters

entities	Constant reference to the entity vector.
idxs	Vector of 1-based indices to be sorted in-place.

Chapter 6

Class Documentation

6.1 core::Device Struct Reference

Represents an IoT device requesting computational resources for a specific service.

`#include <mmkp.hpp>`

Public Member Functions

- Device ()

  Default constructor initializing an empty/placeholder device.
- Device (int id_, double cost_, double cpu_, double mem_, double sto_, int tasks_, int service_)

  Parameterized constructor initializing a device with specific resource demands.

Public Attributes

- int id

  Unique 1-based identifier of the device.
- double cost

  SLA penalty cost incurred if the device is not served.
- double cpu

  Required processing capacity (CPU).
- double mem

  Required memory capacity (MEM).
- double sto

  Required storage capacity (STO).
- int tasks

  Number of atomic tasks composing the requested service.
- int service

  Identifier of the service template assigned to this device.
- bool served = false

  Flag indicating whether the device is currently allocated to a server.
- int server_id = -1

  ID of the server hosting this device (`-1` if unserved).

6.1.1 Detailed Description

Represents an IoT device requesting computational resources for a specific service.

6.1.2 Constructor & Destructor Documentation

6.1.2.1 Device() [1/2]

```
core::Device::Device ( )  [inline]
```

Default constructor initializing an empty/placeholder device.

6.1.2.2 Device() [2/2]

```
core::Device::Device (
            int id_,
            double cost_,
            double cpu_,
            double mem_,
            double sto_,
            int tasks_,
            int service_ )  [inline]
```

Parameterized constructor initializing a device with specific resource demands.

Parameters

*id_*	Unique identifier of the device.
*cost_*	Penalty cost if unserved.
*cpu_*	CPU demand.
*mem_*	Memory demand.
*sto_*	Storage demand.
*tasks_*	Number of service tasks.
*service_*	Service profile ID.

6.1.3 Member Data Documentation

6.1.3.1 cost

```
double core::Device::cost
```

SLA penalty cost incurred if the device is not served.

6.1.3.2 cpu

```
double core::Device::cpu
```

Required processing capacity (CPU).

6.1.3.3 id

```
int core::Device::id
```

Unique 1-based identifier of the device.

6.1.3.4 mem

```
double core::Device::mem
```

Required memory capacity (MEM).

6.1.3.5 served

```
bool core::Device::served = false
```

Flag indicating whether the device is currently allocated to a server.

6.1.3.6 server_id

```
int core::Device::server_id = -1
```

ID of the server hosting this device (`-1` if unserved).

6.1.3.7 service

```
int core::Device::service
```

Identifier of the service template assigned to this device.

6.1.3.8 sto

```
double core::Device::sto
```

Required storage capacity (STO).

6.1.3.9 tasks

```
int core::Device::tasks
```

Number of atomic tasks composing the requested service.

The documentation for this struct was generated from the following file:

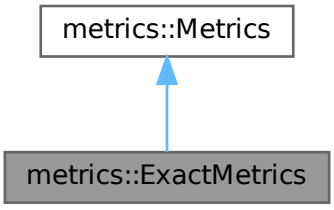
- include/mmkp.hpp

6.2 metrics::ExactMetrics Struct Reference

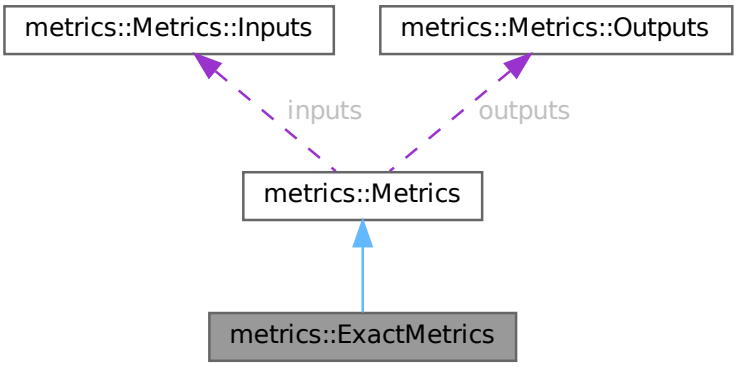
Extends `Metrics` to include CPLEX solver status, objective value, and MIP optimality gap.

```
#include <mmkp.hpp>
```

Inheritance diagram for metrics::ExactMetrics:



Collaboration diagram for metrics::ExactMetrics:



Public Member Functions

- ExactMetrics (Algorithm algorithm_name_, int run_id_, const std::shared_ptr< Metrics > &base)

  Constructs an `ExactMetrics` object copying input dimensions from a base instance.
- ExactMetrics (Algorithm algorithm_name_, int run_id, int d, int s_ec, int s_cc)

  Constructs an `ExactMetrics` object with explicit instance dimensions.
- ExactMetrics (const std::shared_ptr< Metrics > &base, int run_id)

  Constructs an `ExactMetrics` object from a base pointer and run ID.
- std::shared_ptr< Metrics > clone () const override

  Clones this `ExactMetrics` instance polymorphically.
- std::vector< MetricEntry > build_metrics () const override

  Appends CPLEX solver statistics (`Status`, `OF`, `GAP`) to the base metric entries.
- std::vector< std::string > get_header () const override

  Appends exact solver column headers (`Status`, `Obj_Function`, `Gap`) to the CSV header.
- std::vector< std::string > data () const override

  Appends exact solver values to the CSV row data.

Public Member Functions inherited from metrics::Metrics

- Metrics (Paradigm paradigm_, Algorithm algorithm_name_, int run_id)

  Constructs a `Metrics` object with paradigm, algorithm, and run ID.
- Metrics (Paradigm paradigm_, Algorithm algorithm_name_, int run_id, const std::shared_ptr< Metrics > &base)

  Constructs a `Metrics` object copying input dimensions from a base instance.
- Metrics (Paradigm paradigm_, Algorithm algorithm_name_, int run_id, int d, int s_ec, int s_cc)

  Constructs a `Metrics` object with explicit instance counts.
- Metrics (const std::shared_ptr< Metrics > &base, int run_id)

  Copy-like constructor initializing from an existing `Metrics` pointer and updating `run_id`.
- virtual ~Metrics ()=default

  Virtual destructor for safe polymorphic cleanup.
- virtual std::filesystem::path get_base_directory_path () const

  Returns the output directory path corresponding to the current paradigm.
- void save_to_file () const

  Appends the current simulation metrics row to the corresponding algorithm CSV file.

Public Attributes

- std::string status = "Unknown"

  Termination status reported by CPLEX (e.g., `"Optimal"`).
- double objective_function = 0.0

  Objective function value achieved by the solver.
- double gap = 1.0

  Relative MIP optimality gap.

Public Attributes inherited from metrics::Metrics

- Paradigm paradigm = Paradigm::NONE

  Optimization paradigm of the current run.
- Algorithm algorithm = Algorithm::NONE

  Algorithm executed in the current run.
- int run_id = 0

  Execution run identifier (1-indexed).
- struct metrics::Metrics::Inputs inputs

  Instance input parameters.
- struct metrics::Metrics::Outputs outputs

  Simulation output metrics.

6.2.1 Detailed Description

Extends `Metrics` to include CPLEX solver status, objective value, and MIP optimality gap.

6.2.2 Constructor & Destructor Documentation

6.2.2.1 ExactMetrics() [1/3]

```
metrics::ExactMetrics::ExactMetrics (
            Algorithm algorithm_name_,
            int run_id_,
            const std::shared_ptr< Metrics > & base )  [inline]
```

Constructs an `ExactMetrics` object copying input dimensions from a base instance.

Parameters

*algorithm_name_*	Exact algorithm identifier.
*run_id_*	Execution run number.
base	Base metrics pointer containing input parameters.

6.2.2.2 ExactMetrics() [2/3]

```
metrics::ExactMetrics::ExactMetrics (
            Algorithm algorithm_name_,
            int run_id,
            int d,
            int s_ec,
            int s_cc )  [inline]
```

Constructs an `ExactMetrics` object with explicit instance dimensions.

Parameters

*algorithm_name_*	Exact algorithm identifier.
run_id	Execution run number.
d	Total device count.
s_ec	Edge server count.
s_cc	Cloud server count.

6.2.2.3 ExactMetrics() [3/3]

```
metrics::ExactMetrics::ExactMetrics (
            const std::shared_ptr< Metrics > & base,
            int run_id )  [inline]
```

Constructs an `ExactMetrics` object from a base pointer and run ID.

Parameters

base	Base metrics pointer.
run_id	Execution run number.

6.2.3 Member Function Documentation

6.2.3.1 build_metrics()

```
std::vector< MetricEntry > metrics::ExactMetrics::build_metrics ( ) const  [inline], [override],
[virtual]
```

Appends CPLEX solver statistics (`Status`, `OF`, `GAP`) to the base metric entries.

Returns

Vector of `MetricEntry` records.

Reimplemented from metrics::Metrics.

6.2.3.2 clone()

```
std::shared_ptr< Metrics > metrics::ExactMetrics::clone ( ) const  [inline], [override], [virtual]
```

Clones this `ExactMetrics` instance polymorphically.

Returns

Shared pointer to the cloned `ExactMetrics` object.

Reimplemented from metrics::Metrics.

6.2.3.3 data()

```
std::vector< std::string > metrics::ExactMetrics::data ( ) const  [inline], [override], [virtual]
```

Appends exact solver values to the CSV row data.

Returns

Vector of formatted string values.

Reimplemented from metrics::Metrics.

6.2.3.4 get_header()

```
std::vector< std::string > metrics::ExactMetrics::get_header ( ) const  [inline], [override],
[virtual]
```

Appends exact solver column headers (`Status`, `Obj_Function`, `Gap`) to the CSV header.

Returns

Vector of column header strings.

Reimplemented from metrics::Metrics.

6.2.4 Member Data Documentation

6.2.4.1 gap

```
double metrics::ExactMetrics::gap = 1.0
```

Relative MIP optimality gap.

6.2.4.2 objective_function

```
double metrics::ExactMetrics::objective_function = 0.0
```

Objective function value achieved by the solver.

6.2.4.3 status

```
std::string metrics::ExactMetrics::status = "Unknown"
```

Termination status reported by CPLEX (e.g., `"Optimal"`).

The documentation for this struct was generated from the following file:

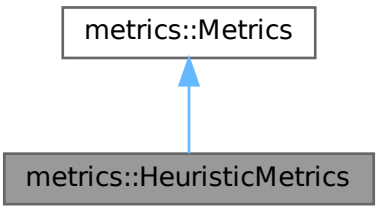
- include/mmkp.hpp

6.3 metrics::HeuristicMetrics Struct Reference

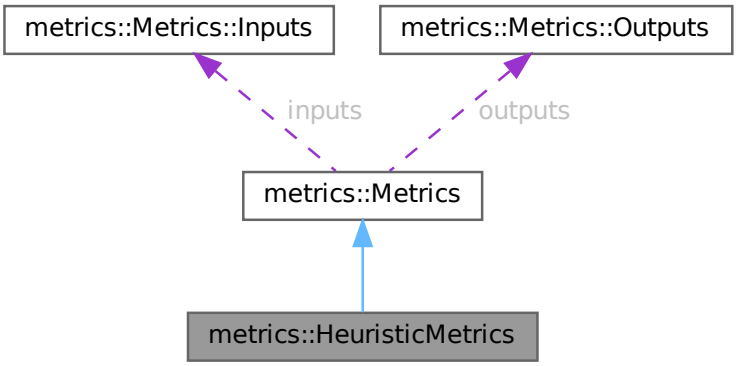
Specialization of `Metrics` for constructive heuristic runs.

`#include <mmkp.hpp>`

Inheritance diagram for metrics::HeuristicMetrics:



Collaboration diagram for metrics::HeuristicMetrics:



Public Member Functions

- HeuristicMetrics (Algorithm algorithm_name_, int run_id_, const std::shared_ptr< Metrics > &base)

  Constructs a `HeuristicMetrics` object copying input dimensions from a base instance.
- HeuristicMetrics (Algorithm algorithm_name_, int run_id, int d, int s_ec, int s_cc)

  Constructs a `HeuristicMetrics` object with explicit instance dimensions.
- HeuristicMetrics (const std::shared_ptr< Metrics > &base, int run_id)

  Constructs a `HeuristicMetrics` object from a base pointer and run ID.
- std::shared_ptr< Metrics > clone () const override

  Clones this `HeuristicMetrics` instance polymorphically.

Public Member Functions inherited from metrics::Metrics

- Metrics (Paradigm paradigm_, Algorithm algorithm_name_, int run_id)

  Constructs a `Metrics` object with paradigm, algorithm, and run ID.
- Metrics (Paradigm paradigm_, Algorithm algorithm_name_, int run_id, const std::shared_ptr< Metrics > &base)

  Constructs a `Metrics` object copying input dimensions from a base instance.
- Metrics (Paradigm paradigm_, Algorithm algorithm_name_, int run_id, int d, int s_ec, int s_cc)

  Constructs a `Metrics` object with explicit instance counts.
- Metrics (const std::shared_ptr< Metrics > &base, int run_id)

  Copy-like constructor initializing from an existing `Metrics` pointer and updating `run_id`.
- virtual ~Metrics ()=default

  Virtual destructor for safe polymorphic cleanup.
- virtual std::filesystem::path get_base_directory_path () const

  Returns the output directory path corresponding to the current paradigm.
- virtual std::vector< MetricEntry > build_metrics () const

  Builds the list of formatted metric entries for console and file reporting.
- virtual std::vector< std::string > get_header () const

  Returns the CSV header columns for the current metric type.
- virtual std::vector< std::string > data () const

  Collects the metric values matching the columns of `get_header()`.
- void save_to_file () const

  Appends the current simulation metrics row to the corresponding algorithm CSV file.

Additional Inherited Members

Public Attributes inherited from metrics::Metrics

- Paradigm paradigm = Paradigm::NONE

  Optimization paradigm of the current run.
- Algorithm algorithm = Algorithm::NONE

  Algorithm executed in the current run.
- int run_id = 0

  Execution run identifier (1-indexed).
- struct metrics::Metrics::Inputs inputs

  Instance input parameters.
- struct metrics::Metrics::Outputs outputs

  Simulation output metrics.

6.3.1 Detailed Description

Specialization of `Metrics` for constructive heuristic runs.

6.3.2 Constructor & Destructor Documentation

6.3.2.1 HeuristicMetrics() [1/3]

```
metrics::HeuristicMetrics::HeuristicMetrics (
            Algorithm algorithm_name_,
            int run_id_,
            const std::shared_ptr< Metrics > & base )  [inline]
```

Constructs a `HeuristicMetrics` object copying input dimensions from a base instance.

Parameters

*algorithm_name_*	Heuristic algorithm identifier.
*run_id_*	Execution run number.
base	Base metrics pointer containing input parameters.

6.3.2.2 HeuristicMetrics() [2/3]

```
metrics::HeuristicMetrics::HeuristicMetrics (
            Algorithm algorithm_name_,
            int run_id,
            int d,
            int s_ec,
            int s_cc )  [inline]
```

Constructs a HeuristicMetrics object with explicit instance dimensions.

Parameters

*algorithm_name_*	Heuristic algorithm identifier.
run_id	Execution run number.
d	Total device count.
s_ec	Edge server count.
s_cc	Cloud server count.

6.3.2.3 HeuristicMetrics() [3/3]

```
metrics::HeuristicMetrics::HeuristicMetrics (
            const std::shared_ptr< Metrics > & base,
            int run_id )  [inline]
```

Constructs a HeuristicMetrics object from a base pointer and run ID.

Parameters

base	Base metrics pointer.
run_id	Execution run number.

6.3.3 Member Function Documentation

6.3.3.1 clone()

```
std::shared_ptr< Metrics > metrics::HeuristicMetrics::clone ( ) const  [inline], [override],
[virtual]
```

Clones this HeuristicMetrics instance polymorphically.

Returns

Shared pointer to the cloned `HeuristicMetrics` object.

Reimplemented from metrics::Metrics.

The documentation for this struct was generated from the following file:

- include/mmkp.hpp

6.4 metrics::Metrics::Inputs Struct Reference

Captures the static input instance dimensions and maximum possible penalty cost.

`#include <mmkp.hpp>`

Public Member Functions

- Inputs & operator= (const Inputs &other)=default

Public Attributes

- int devices_count = 0
  Total number of IoT devices in the instance.
- int servers_ec_count = 0
  Total number of Edge Computing (EC) servers.
- int servers_cc_count = 0
  Total number of Cloud Computing (CC) servers.
- double max_penalty_cost = 0
  Sum of penalty costs across all devices if none are served.

6.4.1 Detailed Description

Captures the static input instance dimensions and maximum possible penalty cost.

6.4.2 Member Function Documentation

6.4.2.1 operator=()

```
Inputs & metrics::Metrics::Inputs::operator= (
            const Inputs & other )  [default]
```

6.4.3 Member Data Documentation

6.4.3.1 devices_count

```
int metrics::Metrics::Inputs::devices_count = 0
```

Total number of IoT devices in the instance.

6.4.3.2 max_penalty_cost

```
double metrics::Metrics::Inputs::max_penalty_cost = 0
```

Sum of penalty costs across all devices if none are served.

6.4.3.3 servers_cc_count

```
int metrics::Metrics::Inputs::servers_cc_count = 0
```

Total number of Cloud Computing (CC) servers.

6.4.3.4 servers_ec_count

```
int metrics::Metrics::Inputs::servers_ec_count = 0
```

Total number of Edge Computing (EC) servers.

The documentation for this struct was generated from the following file:

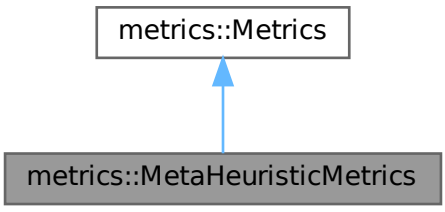
- include/mmkp.hpp

6.5 metrics::MetaHeuristicMetrics Struct Reference

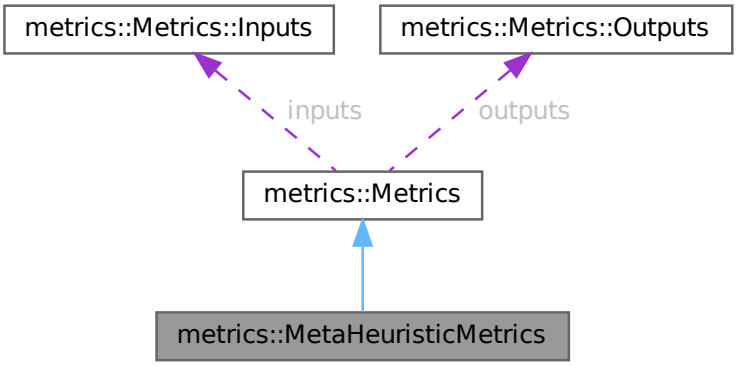
Extends `Metrics` to track Simulated Annealing hyperparameters and initial heuristic selection.

```
#include <mmkp.hpp>
```

Inheritance diagram for metrics::MetaHeuristicMetrics:



Collaboration diagram for metrics::MetaHeuristicMetrics:



Public Member Functions

- MetaHeuristicMetrics (Algorithm algorithm_name_, int run_id_, const std::shared_ptr< Metrics > &base, double temperature_, double alpha_, Algorithm heuristic_)

  Constructs a `MetaHeuristicMetrics` object copying inputs from a base instance.
- MetaHeuristicMetrics (Algorithm algorithm_name_, int run_id, int d, int s_ec, int s_cc, double temperature_, double alpha_, Algorithm heuristic_)

  Constructs a `MetaHeuristicMetrics` object with explicit instance dimensions and hyperparameters.
- MetaHeuristicMetrics (const std::shared_ptr< Metrics > &base, int run_id)

  Constructs a `MetaHeuristicMetrics` object from a base pointer and run ID.
- std::shared_ptr< Metrics > clone () const override

  Clones this `MetaHeuristicMetrics` instance polymorphically.
- std::vector< MetricEntry > build_metrics () const override

  Appends meta-heuristic parameters (`Heuristic`, `Temperature`, `Alpha`) to the metric entries.
- std::vector< std::string > get_header () const override

  Appends meta-heuristic column headers (`Initial_Heuristic`, `Temperature`, `Alpha`) to the CSV header.
- std::vector< std::string > data () const override

  Appends meta-heuristic hyperparameter values to the CSV row data.

Public Member Functions inherited from metrics::Metrics

- Metrics (Paradigm paradigm_, Algorithm algorithm_name_, int run_id)

  Constructs a `Metrics` object with paradigm, algorithm, and run ID.
- Metrics (Paradigm paradigm_, Algorithm algorithm_name_, int run_id, const std::shared_ptr< Metrics > &base)

  Constructs a `Metrics` object copying input dimensions from a base instance.
- Metrics (Paradigm paradigm_, Algorithm algorithm_name_, int run_id, int d, int s_ec, int s_cc)

  Constructs a `Metrics` object with explicit instance counts.
- Metrics (const std::shared_ptr< Metrics > &base, int run_id)

  Copy-like constructor initializing from an existing `Metrics` pointer and updating `run_id`.
- virtual ~Metrics ()=default

Virtual destructor for safe polymorphic cleanup.

- virtual std::filesystem::path get_base_directory_path () const

  Returns the output directory path corresponding to the current paradigm.

- void save_to_file () const

  Appends the current simulation metrics row to the corresponding algorithm CSV file.

Public Attributes

- Algorithm heuristic = Algorithm::NONE

  Constructive heuristic used to generate the initial solution.

- double temperature = 0.0

  Initial temperature T_0 for Simulated Annealing.

- double alpha = 0.0

  Cooling rate factor α for Simulated Annealing.

Public Attributes inherited from metrics::Metrics

- Paradigm paradigm = Paradigm::NONE

  Optimization paradigm of the current run.

- Algorithm algorithm = Algorithm::NONE

  Algorithm executed in the current run.

- int run_id = 0

  Execution run identifier (1-indexed).

- struct metrics::Metrics::Inputs inputs

  Instance input parameters.

- struct metrics::Metrics::Outputs outputs

  Simulation output metrics.

6.5.1 Detailed Description

Extends `Metrics` to track Simulated Annealing hyperparameters and initial heuristic selection.

6.5.2 Constructor & Destructor Documentation

6.5.2.1 MetaHeuristicMetrics() [1/3]

```
metrics::MetaHeuristicMetrics::MetaHeuristicMetrics (
            Algorithm algorithm_name_,
            int run_id_,
            const std::shared_ptr< Metrics > & base,
            double temperature_,
            double alpha_,
            Algorithm heuristic_ )  [inline]
```

Constructs a `MetaHeuristicMetrics` object copying inputs from a base instance.

Parameters

*algorithm_↩ name_*	Meta-heuristic algorithm identifier.
*run_id_*	Execution run number.
base	Base metrics pointer containing input parameters.
*temperature_*	Initial temperature T_0.
*alpha_*	Geometric cooling factor α.
*heuristic_*	Initial constructive heuristic algorithm.

6.5.2.2 MetaHeuristicMetrics() [2/3]

```
metrics::MetaHeuristicMetrics::MetaHeuristicMetrics (
            Algorithm algorithm_name_,
            int run_id,
            int d,
            int s_ec,
            int s_cc,
            double temperature_,
            double alpha_,
            Algorithm heuristic_ )  [inline]
```

Constructs a MetaHeuristicMetrics object with explicit instance dimensions and hyperparameters.

Parameters

*algorithm_↩ name_*	Meta-heuristic algorithm identifier.
run_id	Execution run number.
d	Total device count.
s_ec	Edge server count.
s_cc	Cloud server count.
*temperature_*	Initial temperature T_0.
*alpha_*	Geometric cooling factor α.
*heuristic_*	Initial constructive heuristic algorithm.

6.5.2.3 MetaHeuristicMetrics() [3/3]

```
metrics::MetaHeuristicMetrics::MetaHeuristicMetrics (
            const std::shared_ptr< Metrics > & base,
            int run_id )  [inline]
```

Constructs a MetaHeuristicMetrics object from a base pointer and run ID.

Parameters

base	Base metrics pointer.
run↩ _id	Execution run number.

6.5.3 Member Function Documentation

6.5.3.1 build_metrics()

```
std::vector< MetricEntry > metrics::MetaHeuristicMetrics::build_metrics ( ) const  [inline], [override], [virtual]
```

Appends meta-heuristic parameters (`Heuristic`, `Temperature`, `Alpha`) to the metric entries.

Returns

Vector of `MetricEntry` records.

Reimplemented from metrics::Metrics.

6.5.3.2 clone()

```
std::shared_ptr< Metrics > metrics::MetaHeuristicMetrics::clone ( ) const  [inline], [override], [virtual]
```

Clones this `MetaHeuristicMetrics` instance polymorphically.

Returns

Shared pointer to the cloned `MetaHeuristicMetrics` object.

Reimplemented from metrics::Metrics.

6.5.3.3 data()

```
std::vector< std::string > metrics::MetaHeuristicMetrics::data ( ) const  [inline], [override], [virtual]
```

Appends meta-heuristic hyperparameter values to the CSV row data.

Returns

Vector of formatted string values.

Reimplemented from metrics::Metrics.

6.5.3.4 get_header()

```
std::vector< std::string > metrics::MetaHeuristicMetrics::get_header ( ) const  [inline], [override], [virtual]
```

Appends meta-heuristic column headers (`Initial_Heuristic`, `Temperature`, `Alpha`) to the CSV header.

Returns

Vector of column header strings.

Reimplemented from metrics::Metrics.

6.5.4 Member Data Documentation

6.5.4.1 alpha

```
double metrics::MetaHeuristicMetrics::alpha = 0.0
```

Cooling rate factor α for Simulated Annealing.

6.5.4.2 heuristic

```
Algorithm metrics::MetaHeuristicMetrics::heuristic = Algorithm::NONE
```

Constructive heuristic used to generate the initial solution.

6.5.4.3 temperature

```
double metrics::MetaHeuristicMetrics::temperature = 0.0
```

Initial temperature T_0 for Simulated Annealing.

The documentation for this struct was generated from the following file:

- include/mmkp.hpp

6.6 metrics::MetricEntry Struct Reference

Represents a formatted key-value entry for displaying or exporting simulation metrics.

```
#include <mmkp.hpp>
```

Public Attributes

- std::string category
  Section category header (e.g., "GENERAL", "DEVICES", "COSTS").
- std::string header
  Short column header name for CSV output.
- std::string label
  Human-readable row label for console/ASCII table output.
- std::string file_value
  Raw formatted string value for CSV persistence.
- std::string console_value
  Decorated string value (with units/percentages) for ASCII tables.

6.6.1 Detailed Description

Represents a formatted key-value entry for displaying or exporting simulation metrics.

6.6.2 Member Data Documentation

6.6.2.1 category

```
std::string metrics::MetricEntry::category
```

Section category header (e.g., "GENERAL", "DEVICES", "COSTS").

6.6.2.2 console_value

```
std::string metrics::MetricEntry::console_value
```

Decorated string value (with units/percentages) for ASCII tables.

6.6.2.3 file_value

```
std::string metrics::MetricEntry::file_value
```

Raw formatted string value for CSV persistence.

6.6.2.4 header

```
std::string metrics::MetricEntry::header
```

Short column header name for CSV output.

6.6.2.5 label

```
std::string metrics::MetricEntry::label
```

Human-readable row label for console/ASCII table output.

The documentation for this struct was generated from the following file:

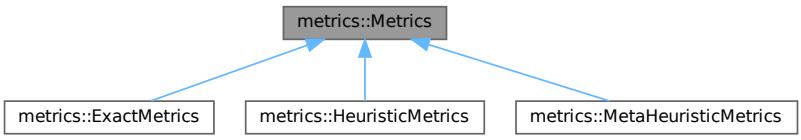
- include/mmkp.hpp

6.7 metrics::Metrics Struct Reference

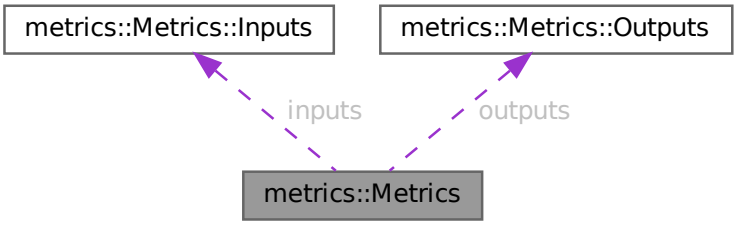
Polymorphic base structure for tracking, calculating, and saving simulation metrics.

`#include <mmkp.hpp>`

Inheritance diagram for metrics::Metrics:



Collaboration diagram for metrics::Metrics:



Classes

- struct Inputs

  Captures the static input instance dimensions and maximum possible penalty cost.
- struct Outputs

  Captures the resulting performance and resource utilization metrics after allocation.

Public Member Functions

- Metrics (Paradigm paradigm_, Algorithm algorithm_name_, int run_id)

  Constructs a `Metrics` object with paradigm, algorithm, and run ID.
- Metrics (Paradigm paradigm_, Algorithm algorithm_name_, int run_id, const std::shared_ptr< Metrics > &base)

  Constructs a `Metrics` object copying input dimensions from a base instance.
- Metrics (Paradigm paradigm_, Algorithm algorithm_name_, int run_id, int d, int s_ec, int s_cc)

  Constructs a `Metrics` object with explicit instance counts.
- Metrics (const std::shared_ptr< Metrics > &base, int run_id)

Copy-like constructor initializing from an existing `Metrics` pointer and updating `run_id`.

- virtual ~Metrics ()=default

  Virtual destructor for safe polymorphic cleanup.
- virtual std::shared_ptr< Metrics > clone () const

  Creates a polymorphic deep copy of this `Metrics` instance.
- virtual std::filesystem::path get_base_directory_path () const

  Returns the output directory path corresponding to the current paradigm.
- virtual std::vector< MetricEntry > build_metrics () const

  Builds the list of formatted metric entries for console and file reporting.
- virtual std::vector< std::string > get_header () const

  Returns the CSV header columns for the current metric type.
- virtual std::vector< std::string > data () const

  Collects the metric values matching the columns of `get_header()`.
- void save_to_file () const

  Appends the current simulation metrics row to the corresponding algorithm CSV file.

Public Attributes

- Paradigm paradigm = Paradigm::NONE

  Optimization paradigm of the current run.
- Algorithm algorithm = Algorithm::NONE

  Algorithm executed in the current run.
- int run_id = 0

  Execution run identifier (1-indexed).
- struct metrics::Metrics::Inputs inputs

  Instance input parameters.
- struct metrics::Metrics::Outputs outputs

  Simulation output metrics.

6.7.1 Detailed Description

Polymorphic base structure for tracking, calculating, and saving simulation metrics.

6.7.2 Constructor & Destructor Documentation

6.7.2.1 Metrics() [1/4]

```
metrics::Metrics::Metrics (
            Paradigm paradigm_,
            Algorithm algorithm_name_,
            int run_id )  [inline]
```

Constructs a `Metrics` object with paradigm, algorithm, and run ID.

Parameters

*paradigm_*	Optimization paradigm.
*algorithm_↩ name_*	Executed algorithm.
run_id	Current run number.

6.7.2.2 Metrics() [2/4]

```
metrics::Metrics::Metrics (
            Paradigm paradigm_,
            Algorithm algorithm_name_,
            int run_id,
            const std::shared_ptr< Metrics > & base )  [inline]
```

Constructs a Metrics object copying input dimensions from a base instance.

Parameters

*paradigm_*	Optimization paradigm.
*algorithm_↩ name_*	Executed algorithm.
run_id	Current run number.
base	Shared pointer to a base Metrics instance containing input parameters.

6.7.2.3 Metrics() [3/4]

```
metrics::Metrics::Metrics (
            Paradigm paradigm_,
            Algorithm algorithm_name_,
            int run_id,
            int d,
            int s_ec,
            int s_cc )  [inline]
```

Constructs a Metrics object with explicit instance counts.

Parameters

*paradigm_*	Optimization paradigm.
*algorithm_↩ name_*	Executed algorithm.
run_id	Current run number.
d	Total device count.
s_ec	Edge server count.
s_cc	Cloud server count.

6.7.2.4 Metrics() [4/4]

```
metrics::Metrics::Metrics (
            const std::shared_ptr< Metrics > & base,
            int run_id )  [inline]
```

Copy-like constructor initializing from an existing Metrics pointer and updating run_id.

Parameters

base	Shared pointer to an existing `Metrics` object.
run_id	New run identifier.

6.7.2.5 ~Metrics()

```
virtual metrics::Metrics::~Metrics ( )  [virtual], [default]
```

Virtual destructor for safe polymorphic cleanup.

6.7.3 Member Function Documentation

6.7.3.1 build_metrics()

```
virtual std::vector< MetricEntry > metrics::Metrics::build_metrics ( ) const  [inline], [virtual]
```

Builds the list of formatted metric entries for console and file reporting.

Returns

Vector of `MetricEntry` records.

Reimplemented in metrics::ExactMetrics, and metrics::MetaHeuristicMetrics.

6.7.3.2 clone()

```
virtual std::shared_ptr< Metrics > metrics::Metrics::clone ( ) const  [inline], [virtual]
```

Creates a polymorphic deep copy of this `Metrics` instance.

Returns

A `std::shared_ptr<Metrics>` pointing to the cloned instance.

Reimplemented in metrics::ExactMetrics, metrics::HeuristicMetrics, and metrics::MetaHeuristicMetrics.

6.7.3.3 data()

```
virtual std::vector< std::string > metrics::Metrics::data ( ) const  [inline], [virtual]
```

Collects the metric values matching the columns of `get_header()`.

Returns

Vector of formatted string values for a single CSV row.

Reimplemented in metrics::ExactMetrics, and metrics::MetaHeuristicMetrics.

6.7.3.4 get_base_directory_path()

```
virtual std::filesystem::path metrics::Metrics::get_base_directory_path ( ) const  [inline],
[virtual]
```

Returns the output directory path corresponding to the current paradigm.

Returns

Filesystem path `results/<PARADIGM>`.

6.7.3.5 get_header()

```
virtual std::vector< std::string > metrics::Metrics::get_header ( ) const  [inline], [virtual]
```

Returns the CSV header columns for the current metric type.

Returns

Vector of column header strings.

Reimplemented in metrics::ExactMetrics, and metrics::MetaHeuristicMetrics.

6.7.3.6 save_to_file()

```
void metrics::Metrics::save_to_file ( ) const  [inline]
```

Appends the current simulation metrics row to the corresponding algorithm CSV file.

6.7.4 Member Data Documentation

6.7.4.1 algorithm

```
Algorithm metrics::Metrics::algorithm = Algorithm::NONE
```

Algorithm executed in the current run.

6.7.4.2 inputs

```
struct metrics::Metrics::Inputs metrics::Metrics::inputs
```

Instance input parameters.

6.7.4.3 outputs

```
struct metrics::Metrics::Outputs metrics::Metrics::outputs
```

Simulation output metrics.

6.7.4.4 paradigm

```
Paradigm metrics::Metrics::paradigm = Paradigm::NONE
```

Optimization paradigm of the current run.

6.7.4.5 run_id

```
int metrics::Metrics::run_id = 0
```

Execution run identifier (1-indexed).

The documentation for this struct was generated from the following file:

- include/mmkp.hpp

6.8 metaheuristic::anonymous_namespace{metaheuristic.hpp}::Move Struct Reference

Represents a reversible state transition within the neighborhood search.

```
#include <metaheuristic.hpp>
```

Public Attributes

- bool valid = false

  Indicates if a feasible move was found and applied.
- int device_id = -1

  ID of the primary device involved in the move.
- int old_server_id = -1

  Original server ID of the primary device (`-1` if previously unserved).
- int new_server_id = -1

  Target server ID where the primary device was allocated.
- int swapped_id = -1

  ID of the secondary device evicted in a swap (`-1` if not a swap).
- double delta_cost = 0.0

  Net change in total penalty cost resulting from the move.

6.8.1 Detailed Description

Represents a reversible state transition within the neighborhood search.

6.8.2 Member Data Documentation

6.8.2.1 delta_cost

```
double metaheuristic::anonymous_namespace{metaheuristic.hpp}::Move::delta_cost = 0.0
```

Net change in total penalty cost resulting from the move.

6.8.2.2 device_id

```
int metaheuristic::anonymous_namespace{metaheuristic.hpp}::Move::device_id = -1
```

ID of the primary device involved in the move.

6.8.2.3 new_server_id

```
int metaheuristic::anonymous_namespace{metaheuristic.hpp}::Move::new_server_id = -1
```

Target server ID where the primary device was allocated.

6.8.2.4 old_server_id

```
int metaheuristic::anonymous_namespace{metaheuristic.hpp}::Move::old_server_id = -1
```

Original server ID of the primary device (-1 if previously unserved).

6.8.2.5 swapped_id

```
int metaheuristic::anonymous_namespace{metaheuristic.hpp}::Move::swapped_id = -1
```

ID of the secondary device evicted in a swap (-1 if not a swap).

6.8.2.6 valid

```
bool metaheuristic::anonymous_namespace{metaheuristic.hpp}::Move::valid = false
```

Indicates if a feasible move was found and applied.

The documentation for this struct was generated from the following file:

- include/metaheuristic.hpp

6.9 metrics::Metrics::Outputs Struct Reference

Captures the resulting performance and resource utilization metrics after allocation.

```
#include <mmkp.hpp>
```

Public Member Functions

- Outputs & operator= (const Outputs &other)=default

Public Attributes

- double execution_time = 0.0

  Total algorithm runtime in seconds.
- int devices_served_count = 0

  Total number of served devices.
- int devices_served_ec_count = 0

  Number of devices served by Edge (EC) servers.
- int devices_served_cc_count = 0

  Number of devices served by Cloud (CC) servers.
- int servers_used_count = 0

  Total number of active servers hosting at least one device.
- int servers_used_ec_count = 0

  Number of active Edge (EC) servers.
- int servers_used_cc_count = 0

  Number of active Cloud (CC) servers.
- double total_penalty_cost = 0.0

  Total penalty cost of all unserved devices.

6.9.1 Detailed Description

Captures the resulting performance and resource utilization metrics after allocation.

6.9.2 Member Function Documentation

6.9.2.1 operator=()

```
Outputs & metrics::Metrics::Outputs::operator= (
            const Outputs & other )  [default]
```

6.9.3 Member Data Documentation

6.9.3.1 devices_served_cc_count

```
int metrics::Metrics::Outputs::devices_served_cc_count = 0
```

Number of devices served by Cloud (CC) servers.

6.9.3.2 devices_served_count

```
int metrics::Metrics::Outputs::devices_served_count = 0
```

Total number of served devices.

6.9.3.3 devices_served_ec_count

```
int metrics::Metrics::Outputs::devices_served_ec_count = 0
```

Number of devices served by Edge (EC) servers.

6.9.3.4 execution_time

```
double metrics::Metrics::Outputs::execution_time = 0.0
```

Total algorithm runtime in seconds.

6.9.3.5 servers_used_cc_count

```
int metrics::Metrics::Outputs::servers_used_cc_count = 0
```

Number of active Cloud (CC) servers.

6.9.3.6 servers_used_count

```
int metrics::Metrics::Outputs::servers_used_count = 0
```

Total number of active servers hosting at least one device.

6.9.3.7 servers_used_ec_count

```
int metrics::Metrics::Outputs::servers_used_ec_count = 0
```

Number of active Edge (EC) servers.

6.9.3.8 total_penalty_cost

```
double metrics::Metrics::Outputs::total_penalty_cost = 0.0
```

Total penalty cost of all unserved devices.

The documentation for this struct was generated from the following file:

- include/mmkp.hpp

6.10 metrics::Result Struct Reference

Encapsulates the complete system state (`Devices`, `Servers`) and polymorphic `Metrics` for a simulation run.

`#include <mmkp.hpp>`

Public Member Functions

- Result (core::Devices d, core::Servers s)

  Constructs a `Result` state from pre-loaded devices and servers vectors.
- Result (const Result &other)

  Deep-copy constructor cloning both entities and the polymorphic `stats` pointer.
- Result (int num_devices, int num_services, int num_ec, int num_cc)

  Loads (or generates) an instance with the specified dimensions and initializes base metrics.
- Result & operator= (const Result &other)

  Deep-copy assignment operator cloning the polymorphic `stats` object.

Public Attributes

- core::Devices devices

  Vector of all IoT devices in the simulation.
- core::Servers servers

  Vector of all Edge and Cloud servers in the simulation.
- std::shared_ptr< Metrics > stats

  Polymorphic pointer to the run's metrics tracker.

6.10.1 Detailed Description

Encapsulates the complete system state (`Devices`, `Servers`) and polymorphic `Metrics` for a simulation run.

6.10.2 Constructor & Destructor Documentation

6.10.2.1 Result() [1/3]

```
metrics::Result::Result (
            core::Devices d,
            core::Servers s )  [inline]
```

Constructs a `Result` state from pre-loaded devices and servers vectors.

Parameters

d	Vector of devices.
s	Vector of servers.

6.10.2.2 Result() [2/3]

```
metrics::Result::Result (
            const Result & other )  [inline]
```

Deep-copy constructor cloning both entities and the polymorphic `stats` pointer.

Parameters

other	Source `Result` instance to copy.

6.10.2.3 Result() [3/3]

```
metrics::Result::Result (
            int num_devices,
            int num_services,
            int num_ec,
            int num_cc )  [inline]
```

Loads (or generates) an instance with the specified dimensions and initializes base metrics.

Parameters

num_devices	Number of IoT devices to load.
num_services	Number of service templates.
num_ec	Number of Edge Computing servers.
num_cc	Number of Cloud Computing servers.

6.10.3 Member Function Documentation

6.10.3.1 operator=()

```
Result & metrics::Result::operator= (
            const Result & other )  [inline]
```

Deep-copy assignment operator cloning the polymorphic `stats` object.

Parameters

other	Source `Result` instance.

Returns

Reference to `*this`.

6.10.4 Member Data Documentation

6.10.4.1 devices

```
core::Devices metrics::Result::devices
```

Vector of all IoT devices in the simulation.

6.10.4.2 servers

```
core::Servers metrics::Result::servers
```

Vector of all Edge and Cloud servers in the simulation.

6.10.4.3 stats

```
std::shared_ptr<Metrics> metrics::Result::stats
```

Polymorphic pointer to the run's metrics tracker.

The documentation for this struct was generated from the following file:

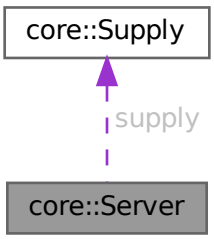
- include/mmkp.hpp

6.11 core::Server Struct Reference

Represents an Edge Computing (EC) or Cloud Computing (CC) server node.

```
#include <mmkp.hpp>
```

Collaboration diagram for core::Server:



Public Member Functions

- Server ()
  Default constructor initializing an empty/placeholder server.
- Server (int id_, char type_, double cpu_, double mem_, double sto_)
  Parameterized constructor initializing a server with given capacities.
- bool can_serve (const Device &device) const
  Checks if the server has enough residual capacity to serve a given device.
- bool add_service (Device &device)
  Allocates a device to this server and updates consumed resources.
- bool rmv_service (Device &device)
  Removes a device from this server and releases its consumed resources.

Public Attributes

- int id

  Unique 1-based identifier of the server.
- char type

  Server tier type ("E' for Edge, 'C' for Cloud).
- double cpu

  Maximum total CPU capacity of the server.
- double mem

  Maximum total memory capacity of the server.
- double sto

  Maximum total storage capacity of the server.
- Supply supply

  Current resource utilization and set of allocated devices.

6.11.1 Detailed Description

Represents an Edge Computing (EC) or Cloud Computing (CC) server node.

6.11.2 Constructor & Destructor Documentation

6.11.2.1 Server() [1/2]

```
core::Server::Server ( )  [inline]
```

Default constructor initializing an empty/placeholder server.

6.11.2.2 Server() [2/2]

```
core::Server::Server (
            int id_,
            char type_,
            double cpu_,
            double mem_,
            double sto_ )  [inline]
```

Parameterized constructor initializing a server with given capacities.

Parameters

*id_*	Unique 1-based server ID.
*type_*	Server tier character ("E' or 'C').
*cpu_*	Total CPU capacity.
*mem_*	Total memory capacity.
*sto_*	Total storage capacity.

6.11.3 Member Function Documentation

6.11.3.1 add_service()

```
bool core::Server::add_service (
            Device & device ) [inline]
```

Allocates a device to this server and updates consumed resources.

Parameters

device	The device to allocate (modified in-place to mark `served = true` and `server_id`).

Returns

`true` if the device was successfully added; `false` if it was already present.

6.11.3.2 can_serve()

```
bool core::Server::can_serve (
            const Device & device ) const [inline]
```

Checks if the server has enough residual capacity to serve a given device.

Parameters

device	The candidate device to be checked.

Returns

`true` if residual CPU, MEM, and STO capacities are sufficient; `false` otherwise.

6.11.3.3 rmv_service()

```
bool core::Server::rmv_service (
            Device & device ) [inline]
```

Removes a device from this server and releases its consumed resources.

Parameters

device	The device to remove (modified in-place to reset `served = false` and `server_id = -1`).

Returns

`true` if the device was successfully removed; `false` if it was not allocated to this server.

6.11.4 Member Data Documentation

6.11.4.1 cpu

```
double core::Server::cpu
```

Maximum total CPU capacity of the server.

6.11.4.2 id

```
int core::Server::id
```

Unique 1-based identifier of the server.

6.11.4.3 mem

```
double core::Server::mem
```

Maximum total memory capacity of the server.

6.11.4.4 sto

```
double core::Server::sto
```

Maximum total storage capacity of the server.

6.11.4.5 supply

```
Supply core::Server::supply
```

Current resource utilization and set of allocated devices.

6.11.4.6 type

```
char core::Server::type
```

Server tier type (‘'E'`for Edge`,'C'` for Cloud).

The documentation for this struct was generated from the following file:

- include/mmkp.hpp

6.12 core::Supply Struct Reference

Tracks the cumulative resource consumption and allocated device IDs on a single server.

```
#include <mmkp.hpp>
```

Public Member Functions

- Supply ()
  Default constructor initializing zero resource usage.
- Supply (double cost_, double cpu_, double mem_, double sto_)
  Parameterized constructor initializing specific resource usage values.

Public Attributes

- double cost = 0.0
  Total avoided penalty cost of devices currently allocated to the server.
- double cpu = 0.0
  Currently consumed CPU capacity on the server.
- double mem = 0.0
  Currently consumed memory capacity on the server.
- double sto = 0.0
  Currently consumed storage capacity on the server.
- i_set devices
  Set of device IDs currently served by the server.

6.12.1 Detailed Description

Tracks the cumulative resource consumption and allocated device IDs on a single server.

6.12.2 Constructor & Destructor Documentation

6.12.2.1 Supply() [1/2]

```
core::Supply::Supply ( )  [inline]
```

Default constructor initializing zero resource usage.

6.12.2.2 Supply() [2/2]

```
core::Supply::Supply (
            double cost_,
            double cpu_,
            double mem_,
            double sto_ )  [inline]
```

Parameterized constructor initializing specific resource usage values.

Parameters

*cost_*	Initial accumulated cost.
*cpu_*	Initial consumed CPU.
*mem_*	Initial consumed memory.
*sto_*	Initial consumed storage.

6.12.3 Member Data Documentation

6.12.3.1 cost

```
double core::Supply::cost = 0.0
```

Total avoided penalty cost of devices currently allocated to the server.

6.12.3.2 cpu

```
double core::Supply::cpu = 0.0
```

Currently consumed CPU capacity on the server.

6.12.3.3 devices

```
i_set core::Supply::devices
```

Set of device IDs currently served by the server.

6.12.3.4 mem

```
double core::Supply::mem = 0.0
```

Currently consumed memory capacity on the server.

6.12.3.5 sto

```
double core::Supply::sto = 0.0
```

Currently consumed storage capacity on the server.

The documentation for this struct was generated from the following file:

- include/mmkp.hpp

Chapter 7

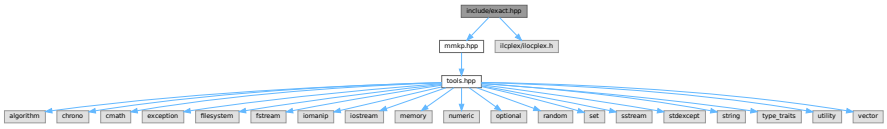
File Documentation

7.1 include/exact.hpp File Reference

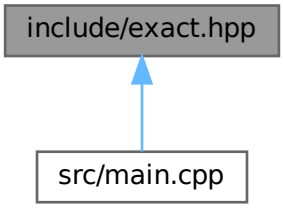
Exact Integer Linear Programming (ILP) formulations using IBM ILOG CPLEX.

```
#include "mmkp.hpp"
#include <ilcplex/ilocplex.h>
```

Include dependency graph for exact.hpp:



This graph shows which files directly or indirectly include this file:



Namespaces

- namespace exact

  Contains mathematical programming models for exact optimization.
- namespace exact::anonymous_namespace{exact.hpp}

Typedefs

- using exact::NumVar2D = IloArray< IloNumVarArray >

  Type alias for a 2D matrix of CPLEX decision variables.

Functions

- void exact::anonymous_namespace{exact.hpp}::solver_maximum_cost (metrics::Result &state, int time_limit_sec=3600)
- void exact::boot (metrics::Result &state, int time_limit_sec=3600)

  Bootstraps and executes the configured exact mathematical model.

7.1.1 Detailed Description

Exact Integer Linear Programming (ILP) formulations using IBM ILOG CPLEX.

Provides the mathematical optimization model and solver configuration to solve the Multiple Multidimensional Knapsack Problem (MMKP) for IoT-EC-CC resource allocation.

7.2 exact.hpp

Go to the documentation of this file.

```
00001
00009 #pragma once
00010
00011 #include "mmkp.hpp"
00012
00013 #include <ilcplex/ilocplex.h>
00014
00022 namespace exact {
00023
00027     using NumVar2D = IloArray<IloNumVarArray>;
00028
00029     namespace {
00030
00041         inline void solver_maximum_cost(metrics::Result& state, int time_limit_sec = 3600) {
00042             core::Devices& devices = state.devices;
00043             core::Servers& servers = state.servers;
00044
00045             auto metrics_ptr = std::dynamic_pointer_cast<metrics::ExactMetrics>(state.stats);
00046             if (!metrics_ptr) {
00047                 std::cerr « "Error: Invalid metrics type for exact solver.\n";
00048                 return;
00049             }
00050
00051             IloEnv env;
00052             try {
00053                 IloModel model(env);
00054                 const size_t num_devices = devices.size();
00055                 const size_t num_servers = servers.size();
00056
00057                 #pragma region VariableDeclaration
00058                 //============================================================================
00059                 // 1. VARIABLE DECLARATION
00060                 //============================================================================
00061
00062                 // x^{i}_{d}: 1 if device d is allocated to server i, 0 otherwise.
00063                 NumVar2D x(env, num_servers);
00064                 for (size_t i = 1; i < num_servers; i++) {
00065                     x[i] = IloNumVarArray(env, num_devices, 0, 1, ILOBOOL);
00066                 }
00067                 #pragma endregion
00068
00069                 #pragma region ObjectiveFunctionAndConstraints
00070                 IloExpr obj(env);
00071
00072                 for (size_t d = 1; d < num_devices; d++) {
```

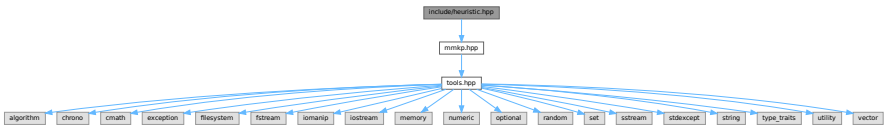
```
00073                     IloExpr c1(env);
00074                     for (size_t i = 1; i < num_servers; i++) {
00075                         c1 += x[i][d];
00076                         obj += devices[d].cost * x[i][d];
00077                     }
00078                     model.add(c1 <= 1);
00079                     c1.end();
00080                 }
00081 
00082                 model.add(IloMaximize(env, obj));
00083                 obj.end();
00084 
00085                 for (size_t i = 1; i < num_servers; i++) {
00086                     IloExpr c_cpu(env), c_mem(env), c_sto(env);
00087 
00088                     for (size_t d = 1; d < num_devices; d++) {
00089                         c_cpu += devices[d].cpu * x[i][d];
00090                         c_mem += devices[d].mem * x[i][d];
00091                         c_sto += devices[d].sto * x[i][d];
00092                     }
00093 
00094                     model.add(c_cpu <= servers[i].cpu);
00095                     model.add(c_mem <= servers[i].mem);
00096                     model.add(c_sto <= servers[i].sto);
00097 
00098                     c_cpu.end();
00099                     c_mem.end();
00100                     c_sto.end();
00101                 }
00102                 #pragma endregion
00103 
00104                 #pragma region SolverConfiguration
00105                 //============================================================================
00106                 // 4. SOLVER CONFIGURATION AND EXECUTION
00107                 //============================================================================
00108 
00109                 IloCplex cplex(model);
00110                 cplex.setOut(env.getNullStream());
00111                 cplex.setWarning(env.getNullStream());
00112 
00113                 cplex.setParam(IloCplex::Param::Threads, 1);
00114                 cplex.setParam(IloCplex::Param::ClockType, 2);
00115                 cplex.setParam(IloCplex::Param::TimeLimit, time_limit_sec);
00116 
00117                 cplex.setParam(IloCplex::Param::WorkMem, 16384);
00118                 cplex.setParam(IloCplex::Param::MIP::Strategy::File, 3);
00119 
00120                 // Relative MIP gap tolerance of 1e-3 (0.001)
00121                 cplex.setParam(IloCplex::Param::MIP::Tolerances::MIPGap, 1e-3);
00122                 cplex.setParam(IloCplex::Param::Emphasis::MIP, IloCplex::MIPEmphasisBalanced);
00123 
00124                 auto start_chrono = std::chrono::high_resolution_clock::now();
00125                 const IloBool has_solution = cplex.solve();
00126                 auto end_chrono = std::chrono::high_resolution_clock::now();
00127 
00128                 metrics_ptr->outputs.execution_time = std::chrono::duration<double>(end_chrono -
    start_chrono).count();
00129 
00130                 std::stringstream status;
00131                 status « cplex.getStatus();
00132                 metrics_ptr->status = status.str();
00133                 #pragma endregion
00134 
00135                 #pragma region Result
00136                 //============================================================================
00137                 // 5. PARSE RESULTS
00138                 //============================================================================
00139 
00140                 if (has_solution) {
00141                     metrics_ptr->objective_function = static_cast<double>(cplex.getObjValue());
00142                     metrics_ptr->gap = static_cast<double>(cplex.getMIPRelativeGap());
00143 
00144                     for (size_t i = 1; i < num_servers; i++) {
00145                         for (size_t d = 1; d < num_devices; d++) {
00146                             if (cplex.isExtracted(x[i][d]) && cplex.getValue(x[i][d]) > 0.5) {
00147                                 servers[i].add_service(devices[d]);
00148                             }
00149                         }
00150                     }
00151                 }
00152 
00153             } catch (const IloException& e) {
00154                 std::cerr « "CPLEX Error: " « e.getMessage() « "\n";
00155             } catch (const std::exception& e) {
00156                 std::cerr « "Standard Exception: " « e.what() « "\n";
00157             } catch (...) {
00158                 std::cerr « "An unknown error occurred in the CPLEX model.\n";
```

```
00159             }
00160             env.end();
00161         }
00162     }
00163
00172     inline void boot(metrics::Result& state, int time_limit_sec = 3600) {
00173         if (!state.stats) return;
00174
00175         if (metrics::Algorithm::solver_maximum_cost == state.stats->algorithm) {
00176             solver_maximum_cost(state, time_limit_sec);
00177         } else {
00178             std::cerr « "Error: Unknown mathematical model algorithm type.\n";
00179         }
00180     }
00181 }
```

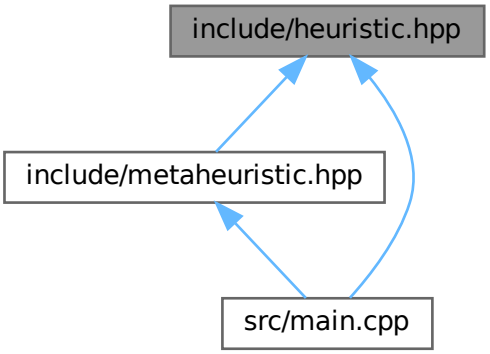
7.3 include/heuristic.hpp File Reference

Constructive heuristic algorithms for fast MMKP resource allocation.

`#include "mmkp.hpp"`
Include dependency graph for heuristic.hpp:



This graph shows which files directly or indirectly include this file:



Namespaces

- namespace heuristic

  Contains constructive heuristic algorithms for network resource allocation.
- namespace heuristic::anonymous_namespace{heuristic.hpp}

Functions

- void heuristic::anonymous_namespace{heuristic.hpp}::greedy_cost_benefit (metrics::Result &state)

  Executes a Global Greedy Resource Allocation based on surrogate pseudo-utility.
- void heuristic::anonymous_namespace{heuristic.hpp}::greedy_random (metrics::Result &state)

  Executes a fast random baseline resource allocation.
- void heuristic::anonymous_namespace{heuristic.hpp}::greedy_random_fallback (metrics::Result &state)

  Executes a two-stage random fallback resource allocation.
- void heuristic::boot (metrics::Result &state)

  Bootstraps and executes the configured constructive heuristic.

7.3.1 Detailed Description

Constructive heuristic algorithms for fast MMKP resource allocation.

Implements deterministic cost-benefit greedy allocation (`greedy_cost_benefit`) as well as stochastic single-stage (`greedy_random`) and two-stage Edge-first (`greedy_random_fallback`) constructive heuristics.

7.4 heuristic.hpp

Go to the documentation of this file.

```
00001
00009 #pragma once
00010
00011 #include "mmkp.hpp"
00012
00020 namespace heuristic {
00021
00022     namespace {
00023
00032         inline void greedy_cost_benefit(metrics::Result& state) {
00033             auto metrics_ptr = std::dynamic_pointer_cast<metrics::HeuristicMetrics>(state.stats);
00034             if (!metrics_ptr) {
00035                 std::cerr « "Error: Invalid metrics type for heuristic solver.\n";
00036                 return;
00037             }
00038
00039             core::Devices& devices = state.devices;
00040             core::Servers& servers = state.servers;
00041
00042             auto start_chrono = std::chrono::high_resolution_clock::now();
00043
00044             double total_cpu = 0.0, total_mem = 0.0, total_sto = 0.0;
00045             for (size_t i = 1; i < servers.size(); i++) {
00046                 total_cpu += servers[i].cpu;
00047                 total_mem += servers[i].mem;
00048                 total_sto += servers[i].sto;
00049             }
00050
00051             id_pair_vector candidates;
00052             candidates.reserve(devices.size());
00053
00054             double min_cpu = std::numeric_limits<double>::max();
00055             double min_mem = std::numeric_limits<double>::max();
00056             double min_sto = std::numeric_limits<double>::max();
00057
00058             for (size_t d = 1; d < devices.size(); d++) {
00059                 core::Device& device = devices[d];
00060                 if (!device.served) {
00061                     double w_cpu = device.cpu / total_cpu;
00062                     double w_mem = device.mem / total_mem;
00063                     double w_sto = device.sto / total_sto;
00064
00065                     double utility = device.cost / (w_cpu + w_mem + w_sto);
00066                     candidates.emplace_back(device.id, utility);
00067
00068                     if (device.cpu < min_cpu) min_cpu = device.cpu;
00069                     if (device.mem < min_mem) min_mem = device.mem;
00070                     if (device.sto < min_sto) min_sto = device.sto;
```

```
00071                 }
00072             }
00073
00074             std::sort(candidates.begin(), candidates.end(), [](const id_pair& a, const id_pair& b) {
00075                 return a.second > b.second;
00076             });
00077
00078             size_t start_idx = 0;
00079             const size_t num_candidates = candidates.size();
00080
00081             for (size_t i = 1; i < servers.size() && start_idx < num_candidates; i++) {
00082                 core::Server& server = servers[i];
00083
00084                 for (size_t c = start_idx; c < num_candidates; c++) {
00085                     if ((server.cpu - server.supply.cpu < min_cpu) ||
00086                         (server.mem - server.supply.mem < min_mem) ||
00087                         (server.sto - server.supply.sto < min_sto)) {
00088                         break;
00089                     }
00090
00091                     core::Device& device = devices[candidates[c].first];
00092                     if (!device.served && server.can_serve(device)) {
00093                         server.add_service(device);
00094                     }
00095
00096                     if (c == start_idx && device.served) {
00097                         start_idx++;
00098                     }
00099                 }
00100             }
00101
00102             auto end_chrono = std::chrono::high_resolution_clock::now();
00103             metrics_ptr->outputs.execution_time = std::chrono::duration<double>(end_chrono -
    start_chrono).count();
00104         }
00105
00114         inline void greedy_random(metrics::Result& state) {
00115             auto metrics_ptr = std::dynamic_pointer_cast<metrics::HeuristicMetrics>(state.stats);
00116             if (!metrics_ptr) {
00117                 std::cerr << "Error: Invalid metrics type for heuristic solver.\n";
00118                 return;
00119             }
00120
00121             core::Devices& devices = state.devices;
00122             core::Servers& servers = state.servers;
00123
00124             int size_servers = static_cast<int>(servers.size()) - 1;
00125
00126             auto start_chrono = std::chrono::high_resolution_clock::now();
00127
00128             for (size_t d = 1; d < devices.size(); d++) {
00129                 core::Device& device = devices[d];
00130
00131                 if (size_servers > 1) {
00132                     int ec_id = randomness::random_number<int>(1, size_servers);
00133                     if (servers[ec_id].can_serve(device)) {
00134                         servers[ec_id].add_service(device);
00135                     }
00136                 }
00137             }
00138
00139             auto end_chrono = std::chrono::high_resolution_clock::now();
00140             metrics_ptr->outputs.execution_time = std::chrono::duration<double>(end_chrono -
    start_chrono).count();
00141         }
00142
00150         inline void greedy_random_fallback(metrics::Result& state) {
00151             auto metrics_ptr = std::dynamic_pointer_cast<metrics::HeuristicMetrics>(state.stats);
00152             if (!metrics_ptr) {
00153                 std::cerr << "Error: Invalid metrics type for heuristic solver.\n";
00154                 return;
00155             }
00156
00157             int ec_count = metrics_ptr->inputs.servers_ec_count;
00158             int cc_count = metrics_ptr->inputs.servers_cc_count;
00159
00160             core::Devices& devices = state.devices;
00161             core::Servers& servers = state.servers;
00162
00163             auto start_chrono = std::chrono::high_resolution_clock::now();
00164
00165             for (size_t d = 1; d < devices.size(); d++) {
00166                 core::Device& device = devices[d];
00167
00168                 if (ec_count > 0) {
00169                     int ec_id = randomness::random_number<int>(1, ec_count);
00170                     if (servers[ec_id].can_serve(device)) {
```

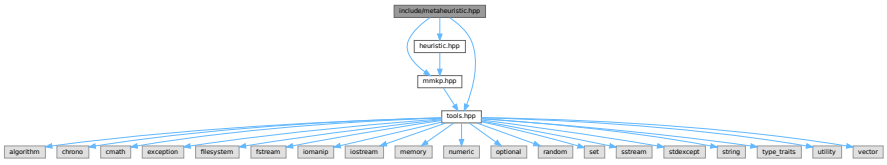
```
00171                         servers[ec_id].add_service(device);
00172                         continue;
00173                     }
00174                 }
00175
00176                 if (cc_count > 0) {
00177                     int cc_id = randomness::random_number<int>(ec_count + 1, ec_count + cc_count);
00178                     if (servers[cc_id].can_serve(device)) {
00179                         servers[cc_id].add_service(device);
00180                     }
00181                 }
00182             }
00183
00184             auto end_chrono = std::chrono::high_resolution_clock::now();
00185             metrics_ptr->outputs.execution_time = std::chrono::duration<double>(end_chrono -
      start_chrono).count();
00186         }
00187     }
00188
00196     inline void boot(metrics::Result& state) {
00197         if (!state.stats) return;
00198
00199         if (metrics::Algorithm::greedy_cost_benefit == state.stats->algorithm) {
00200             greedy_cost_benefit(state);
00201         } else if (metrics::Algorithm::greedy_random == state.stats->algorithm) {
00202             greedy_random(state);
00203         } else if (metrics::Algorithm::greedy_random_fallback == state.stats->algorithm) {
00204             greedy_random_fallback(state);
00205         } else {
00206             std::cerr « "Error: Unknown heuristic algorithm type.\n";
00207         }
00208     }
00209 }
```

7.5 include/metaheuristic.hpp File Reference

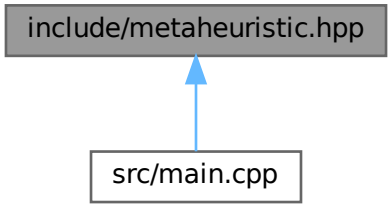
Meta-heuristic optimization algorithms and neighborhood operators for MMKP.

```
#include "mmkp.hpp"
#include "heuristic.hpp"
#include "tools.hpp"
```

Include dependency graph for metaheuristic.hpp:



This graph shows which files directly or indirectly include this file:



Classes

- struct metaheuristic::anonymous_namespace{metaheuristic.hpp}::Move

  Represents a reversible state transition within the neighborhood search.

Namespaces

- namespace metaheuristic

  Encapsulates meta-heuristic algorithms for network resource allocation.
- namespace metaheuristic::anonymous_namespace{metaheuristic.hpp}

Functions

- Move metaheuristic::anonymous_namespace{metaheuristic.hpp}::apply_neighbor (core::Devices &devices, core::Servers &servers, int tries=10)

  Applies a shift, insert, or swap neighborhood move to the current state.
- void metaheuristic::anonymous_namespace{metaheuristic.hpp}::revert_neighbor (core::Devices &devices, core::Servers &servers, const Move &move)

  Reverts a previously applied neighborhood move in $O(\log N)$ time.
- void metaheuristic::anonymous_namespace{metaheuristic.hpp}::simulated_annealing (metrics::Result &state)

  Executes the Simulated Annealing (SA) optimization algorithm.
- void metaheuristic::boot (metrics::Result &state)

  Initializes and launches the selected meta-heuristic algorithm.

Variables

- constexpr int metaheuristic::anonymous_namespace{metaheuristic.hpp}::LOOP = 10

  Number of inner-loop iterations (Markov chain length) executed per temperature level.

7.5.1 Detailed Description

Meta-heuristic optimization algorithms and neighborhood operators for MMKP.

Implements Simulated Annealing (SA) with incremental delta evaluation and shift/insert/swap neighborhood moves to refine initial constructive heuristic solutions.

7.6 metaheuristic.hpp

Go to the documentation of this file.

```
00001
00008 #pragma once
00009
00010 #include "mmkp.hpp"
00011 #include "heuristic.hpp"
00012 #include "tools.hpp"
00013
00021 namespace metaheuristic {
00022
00023     namespace {
00024
00028         constexpr int LOOP = 10;
00029
00034         struct Move {
00035             bool valid = false;
00036             int device_id = -1;
00037             int old_server_id = -1;
00038             int new_server_id = -1;
00039             int swapped_id = -1;
00040             double delta_cost = 0.0;
00041         };
00042
00054         inline Move apply_neighbor(core::Devices& devices, core::Servers& servers, int tries = 10) {
00055             Move move;
00056             const int max_dev = static_cast<int>(devices.size() - 1);
00057             const int max_srv = static_cast<int>(servers.size() - 1);
00058
00059             move.device_id = randomness::random_number<int>(1, max_dev);
00060             core::Device& device = devices[move.device_id];
00061             move.old_server_id = device.server_id;
00062
00063             int count = 0;
00064             while (count < tries) {
00065                 int i = (!device.served && count == tries - 1)
00066                         ? max_srv
00067                         : randomness::random_number<int>(1, max_srv);
00068
00069                 if (i == device.server_id) continue;
00070                 count++;
00071
00072                 core::Server& new_server = servers[i];
00073
00074                 if (new_server.can_serve(device)) {
00075                     if (device.served) {
00076                         servers[move.old_server_id].rmv_service(device);
00077                         move.delta_cost = 0.0;
00078                     } else {
00079                         move.delta_cost = -device.cost;
00080                     }
00081
00082                     new_server.add_service(device);
00083                     move.new_server_id = i;
00084                     move.valid = true;
00085                     return move;
00086                 }
00087             }
00088
00089             if (!device.served) {
00090                 for (int t = 0; t < tries; t++) {
00091                     int cand_id = randomness::random_number<int>(1, max_dev);
00092                     core::Device& cand = devices[cand_id];
00093                     if (!cand.served) continue;
00094
00095                     int srv_id = cand.server_id;
00096                     core::Server& target_server = servers[srv_id];
00097
00098                     target_server.rmv_service(cand);
00099                     if (target_server.can_serve(device)) {
00100                         target_server.add_service(device);
00101                         move.new_server_id = srv_id;
00102                         move.swapped_id = cand_id;
00103                         move.delta_cost = cand.cost - device.cost;
00104                         move.valid = true;
00105                         return move;
00106                     }
00107                     target_server.add_service(cand);
00108                 }
00109             }
00110
00111             return move;
00112         }
00113
```

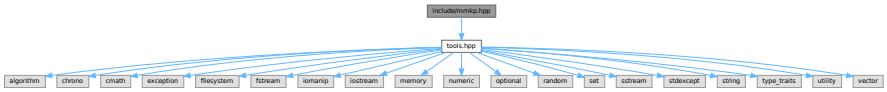
```
00122         inline void revert_neighbor(core::Devices& devices, core::Servers& servers, const Move& move)
      {
00123             core::Device& device = devices[move.device_id];
00124 
00125             if (move.new_server_id != -1) {
00126                 servers[move.new_server_id].rmv_service(device);
00127             }
00128             if (move.swapped_id != -1) {
00129                 servers[move.new_server_id].add_service(devices[move.swapped_id]);
00130             }
00131             if (move.old_server_id != -1) {
00132                 servers[move.old_server_id].add_service(device);
00133             }
00134         }
00135 
00145         inline void simulated_annealing(metrics::Result& state) {
00146             auto metrics_ptr = std::dynamic_pointer_cast<metrics::MetaHeuristicMetrics>(state.stats);
00147             if (!metrics_ptr) {
00148                 std::cerr « "Error: Invalid metrics type for heuristic solver.\n";
00149                 return;
00150             }
00151 
00152             core::Devices best_devices = state.devices;
00153             core::Servers best_servers = state.servers;
00154 
00155             double best_cost = metrics_ptr->outputs.total_penalty_cost;
00156             double current_cost = best_cost;
00157             bool is_best_synced = true;
00158 
00159             double T = metrics_ptr->temperature;
00160             double alpha = metrics_ptr->alpha;
00161 
00162             auto start_chrono = std::chrono::high_resolution_clock::now();
00163 
00164             while (T > 1e-3) {
00165                 for (int i = 0; i < LOOP; i++) {
00166                     Move move = apply_neighbor(state.devices, state.servers);
00167                     if (!move.valid) continue;
00168 
00169                     double neighbor_cost = current_cost + move.delta_cost;
00170                     double delta = neighbor_cost - current_cost;
00171 
00172                     if (delta < 0.0 || (randomness::random_number(0.0, 1.0) < std::exp(-delta / T))) {
00173                         if (delta > 0.0 && !is_best_synced) {
00174                             revert_neighbor(state.devices, state.servers, move);
00175                             best_devices = state.devices;
00176                             best_servers = state.servers;
00177                             is_best_synced = true;
00178                             if (move.swapped_id != -1) {
00179 
      state.servers[move.new_server_id].rmv_service(state.devices[move.swapped_id]);
00180                             }
00181                             if (move.old_server_id != -1) {
00182 
      state.servers[move.old_server_id].rmv_service(state.devices[move.device_id]);
00183                             }
00184                             if (move.new_server_id != -1) {
00185 
      state.servers[move.new_server_id].add_service(state.devices[move.device_id]);
00186                             }
00187                         }
00188 
00189                         current_cost = neighbor_cost;
00190 
00191                         if (current_cost < best_cost - ZERO) {
00192                             i = 0;
00193                             best_cost = current_cost;
00194                             is_best_synced = false;
00195                         }
00196                     } else {
00197                         revert_neighbor(state.devices, state.servers, move);
00198                     }
00199                 }
00200                 T *= alpha;
00201             }
00202 
00203             if (is_best_synced) {
00204                 state.devices = std::move(best_devices);
00205                 state.servers = std::move(best_servers);
00206             }
00207 
00208             auto end_chrono = std::chrono::high_resolution_clock::now();
00209             metrics_ptr->outputs.execution_time += std::chrono::duration<double>(end_chrono -
      start_chrono).count();
00210         }
00211     }
00212 
```

```
00221     inline void boot(metrics::Result& state) {
00222         if (!state.stats) return;
00223 
00224         auto metrics_ptr = std::dynamic_pointer_cast<metrics::MetaHeuristicMetrics>(state.stats);
00225         if (!metrics_ptr) {
00226             std::cerr « "Error: Invalid metrics type for meta-heuristic solver.\n";
00227             return;
00228         }
00229 
00230         metrics::Result initial_state = state;
00231         initial_state.stats = std::make_shared<metrics::HeuristicMetrics>(
00232             metrics_ptr->heuristic,
00233             metrics_ptr->run_id,
00234             state.stats
00235         );
00236 
00237         heuristic::boot(initial_state);
00238         metrics::calculate_metrics(initial_state);
00239         if (metrics_ptr->heuristic != metrics::Algorithm::greedy_cost_benefit || metrics_ptr->run_id
      == 1) {
00240             metrics::save_results(initial_state, false);
00241         }
00242 
00243         state.devices = initial_state.devices;
00244         state.servers = initial_state.servers;
00245         metrics_ptr->inputs = initial_state.stats->inputs;
00246         metrics_ptr->outputs = initial_state.stats->outputs;
00247 
00248         if (metrics_ptr->algorithm == metrics::Algorithm::simulated_annealing) {
00249             simulated_annealing(state);
00250         } else {
00251             std::cerr « "Error: Unknown metaheuristic algorithm type.\n";
00252         }
00253     }
00254 }
```

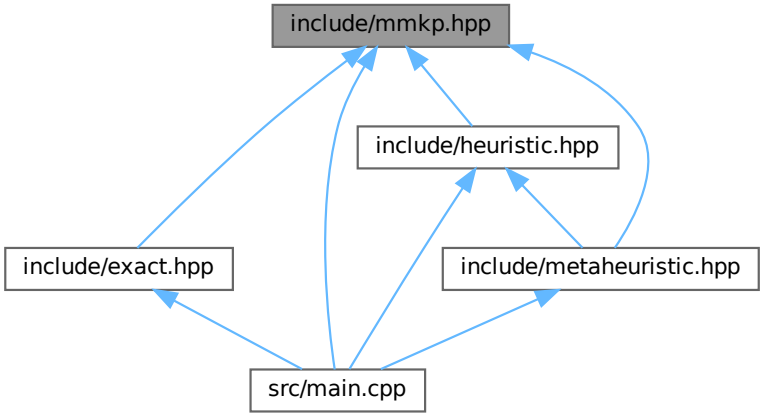
7.7 include/mmkp.hpp File Reference

Core data structures, synthetic dataset generators, metric trackers, and console/file display routines.

`#include "tools.hpp"`
Include dependency graph for mmkp.hpp:



This graph shows which files directly or indirectly include this file:



Classes

- struct core::Device

  Represents an IoT device requesting computational resources for a specific service.
- struct core::Supply

  Tracks the cumulative resource consumption and allocated device IDs on a single server.
- struct core::Server

  Represents an Edge Computing (EC) or Cloud Computing (CC) server node.
- struct metrics::MetricEntry

  Represents a formatted key-value entry for displaying or exporting simulation metrics.
- struct metrics::Metrics

  Polymorphic base structure for tracking, calculating, and saving simulation metrics.
- struct metrics::Metrics::Inputs

  Captures the static input instance dimensions and maximum possible penalty cost.
- struct metrics::Metrics::Outputs

  Captures the resulting performance and resource utilization metrics after allocation.
- struct metrics::ExactMetrics

  Extends `Metrics` to include CPLEX solver status, objective value, and MIP optimality gap.
- struct metrics::HeuristicMetrics

  Specialization of `Metrics` for constructive heuristic runs.
- struct metrics::MetaHeuristicMetrics

  Extends `Metrics` to track Simulated Annealing hyperparameters and initial heuristic selection.
- struct metrics::Result

  Encapsulates the complete system state (`Devices`, `Servers`) and polymorphic `Metrics` for a simulation run.

Namespaces

- namespace core

  Contains the fundamental domain entities for the MMKP resource allocation problem.
- namespace data::generate

  Generates synthetic datasets for services, IoT devices, and EC/CC servers when files are absent.
- namespace data
- namespace data::load

  Parses raw string datasets into strongly-typed `core::Devices` and `core::Servers` vectors.
- namespace metrics

  Provides structures and routines for tracking, formatting, and persisting simulation metrics.
- namespace display

  Utility namespace for formatting and printing entities and ASCII metric tables to console or files.
- namespace display::details

  Internal helper routines for rendering ASCII box-drawing tables.

Typedefs

- using ii_pair = std::pair< int, int >

  Pair of two integers.
- using id_pair = std::pair< int, double >

  Pair of an integer ID and a double value (e.g., utility).
- using i_vector = std::vector< int >

  Vector of integers.
- using s_vector = std::vector< std::string >

  Vector of strings.
- using ii_pair_vector = std::vector< ii_pair >

  Vector of integer-integer pairs.
- using id_pair_vector = std::vector< id_pair >

  Vector of integer-double pairs.
- using i_matrix = std::vector< i_vector >

  2D matrix of integers.
- using s_matrix = std::vector< s_vector >

  2D matrix of strings.
- using i_set = std::set< int >

  Ordered set of unique integers.
- using core::Devices = std::vector< Device >

  Contiguous collection of `Device` instances (1-indexed, index 0 is a placeholder).
- using core::Servers = std::vector< Server >

  Contiguous collection of `Server` instances (1-indexed, index 0 is a placeholder).

Enumerations

- enum class metrics::Paradigm { metrics::NONE , metrics::EXACT , metrics::HEURISTIC , metrics::METAHEURISTIC }

  Enumerates the optimization paradigms supported by the simulator.
- enum class metrics::Algorithm { metrics::NONE , metrics::solver_maximum_cost , metrics::greedy_cost_benefit , metrics::greedy_random , metrics::greedy_random_fallback , metrics::simulated_annealing }

  Identifies the specific optimization algorithm used in a simulation run.

Functions

- std::vector< std::vector< std::string > > data::generate::services (const int num_services=10)

  Generates or loads network service requirement templates.
- std::vector< std::vector< std::string > > data::generate::devices (const int length, const int num_services=10)

  Generates or loads device request data representing IoT network demands.
- std::vector< std::vector< std::string > > data::generate::servers (const int ec_length=99, const int cc_length=1)

  Generates or loads server infrastructure data for Edge and Cloud nodes.
- core::Devices data::load::devices (const int length, const int num_services=10)

  Loads device data from disk (generating it if absent) into a vector of `core::Device` objects.
- core::Servers data::load::servers (const int ec=99, const int cc=1)

  Loads Edge and Cloud server data from disk (generating it if absent).
- std::string metrics::to_string (Paradigm paradigm)

  Converts a `Paradigm` enum value to its uppercase string representation.
- std::string metrics::to_string (Algorithm algorithm)

  Converts an `Algorithm` enum value to its standardized string identifier.
- void metrics::calculate_metrics (Result &state)

  Computes the final simulation metrics based on the current device-to-server allocations.
- void metrics::save_results (const Result &state, bool show_servers_usage=false)

  Saves the final calculated metrics and ASCII summary tables to the file system.
- void display::show_device (const core::Device &device)

  Displays detailed information of a single device to standard output.
- void display::show_devices (const core::Devices &devices)

  Displays detailed information for all devices in the vector.
- void display::show_server (const core::Server &server)

  Displays capacity and current demand of a single server.
- void display::show_servers (const core::Servers &servers)

  Displays capacity and demand information for all servers.
- void display::details::print_row (std::ostream &out, const std::string &label, const std::string &value)

  Prints a formatted key-value row within the ASCII table.
- void display::details::print_header (std::ostream &out)

  Prints the top or bottom border of the ASCII table.
- void display::details::print_middle (std::ostream &out)

  Prints a section separator line within the ASCII table.
- void display::details::print_title (std::ostream &out, const std::string &title)

  Prints a centered title row within the ASCII table.
- void display::show_metrics_console_or_file (const metrics::Metrics &metrics, std::ostream &out)

  Formats and writes the complete simulation metrics table to an output stream.
- void display::show_metrics (const std::shared_ptr< metrics::Metrics > &ptr)

  Displays the metrics table on the console and appends it to a drawing file.
- void display::show_servers_usage (const core::Servers &servers)

  Prints a summary table of server resource utilization percentages.

Variables

- const std::filesystem::path data::generate::DATA_PATH = std::filesystem::current_path() / "data"

  Path pointing to the root `data/` directory.
- const std::filesystem::path metrics::RESULTS_PATH = std::filesystem::current_path() / "results"

  Path pointing to the root `results/` directory.
- const int display::details::total_width = 72

  Total character width of the formatted ASCII table.
- const int display::details::label_width = 26

  Character width allocated to the left-hand label column.

7.7.1 Detailed Description

Core data structures, synthetic dataset generators, metric trackers, and console/file display routines.

Defines the domain entities (`Device`, `Server`, `Supply`), dataset loading/generation utilities, polymorphic metric collection classes (`Metrics`, `ExactMetrics`, `HeuristicMetrics`, `MetaHeuristicMetrics`), and ASCII table visualization functions for the MMKP IoT-EC-CC simulator.

7.7.2 Typedef Documentation

7.7.2.1 i_matrix

```
using i_matrix = std::vector<i_vector>
```

2D matrix of integers.

7.7.2.2 i_set

```
using i_set = std::set<int>
```

Ordered set of unique integers.

7.7.2.3 i_vector

```
using i_vector = std::vector<int>
```

Vector of integers.

7.7.2.4 id_pair

```
using id_pair = std::pair<int, double>
```

Pair of an integer ID and a double value (e.g., utility).

7.7.2.5 id_pair_vector

```
using id_pair_vector = std::vector<id_pair>
```

Vector of integer-double pairs.

7.7.2.6 ii_pair

```
using ii_pair = std::pair<int, int>
```

Pair of two integers.

7.7.2.7 ii_pair_vector

```
using ii_pair_vector = std::vector<ii_pair>
```

Vector of integer-integer pairs.

7.7.2.8 s_matrix

```
using s_matrix = std::vector<s_vector>
```

2D matrix of strings.

7.7.2.9 s_vector

```
using s_vector = std::vector<std::string>
```

Vector of strings.

7.8 mmkp.hpp

Go to the documentation of this file.

```
00001
00009 #pragma once
00010
00011 #include "tools.hpp"
00012
00013 // Type aliases replacing C-style macros
00014 using ii_pair = std::pair<int, int>;
00015 using id_pair = std::pair<int, double>;
00016 using i_vector = std::vector<int>;
00017 using s_vector = std::vector<std::string>;
00018 using ii_pair_vector = std::vector<ii_pair>;
00019 using id_pair_vector = std::vector<id_pair>;
00020 using i_matrix = std::vector<i_vector>;
00021 using s_matrix = std::vector<s_vector>;
00022 using i_set = std::set<int>;
00023
00028 namespace core {
00029
00034     struct Device {
00035         int id;
00036         double cost;
00037         double cpu;
00038         double mem;
00039         double sto;
00040         int tasks;
00041         int service;
00042         bool served = false;
00043         int server_id = -1;
00044
00048         Device() : id(0), cost(0.0), cpu(0.0), mem(0.0), sto(0.0), tasks(0), service(0) {}
00049
00060         Device(int id_, double cost_, double cpu_, double mem_, double sto_, int tasks_, int service_)
00061             : id(id_), cost(cost_), cpu(cpu_), mem(mem_), sto(sto_), tasks(tasks_), service(service_)
      {}
00062     };
00063
00067     using Devices = std::vector<Device>;
00068
00073     struct Supply {
00074         double cost = 0.0;
00075         double cpu = 0.0;
00076         double mem = 0.0;
00077         double sto = 0.0;
00078         //double free = 0.0;
00079         i_set devices;
00080
```

```
00084         Supply() : cost(0.0), cpu(0.0), mem(0.0), sto(0.0) {}
00085
00093         Supply(double cost_,  double cpu_, double mem_, double sto_)
00094             : cost(cost_), cpu(cpu_), mem(mem_), sto(sto_) {}
00095     };
00096
00101     struct Server {
00102         int id;
00103         char type;
00104         double cpu;
00105         double mem;
00106         double sto;
00107         Supply supply;
00108
00112         Server() : id(0), type(' '), cpu(0.0), mem(0.0), sto(0.0), supply() {}
00113
00122         Server(int id_, char type_, double cpu_, double mem_, double sto_)
00123             : id(id_), type(type_), cpu(cpu_), mem(mem_), sto(sto_) {}
00124
00130         inline bool can_serve(const Device& device) const {
00131             return (
00132                 (device.cpu <= (cpu - supply.cpu))  &&
00133                 (device.mem <= (mem - supply.mem))  &&
00134                 (device.sto <= (sto - supply.sto))
00135             );
00136         }
00137
00143         inline bool add_service(Device& device) {
00144             auto result = supply.devices.insert(device.id);
00145             if (!result.second) return false;
00146
00147             device.served = true;
00148             device.server_id = id;
00149
00150             supply.cost += device.cost;
00151             supply.cpu += device.cpu;
00152             supply.mem += device.mem;
00153             supply.sto += device.sto;
00154
00155             return true;
00156         }
00157
00163         inline bool rmv_service(Device& device) {
00164             if (supply.devices.erase(device.id) == 0) return false;
00165
00166             device.served = false;
00167             device.server_id = -1;
00168
00169             supply.cost -= device.cost;
00170             supply.cpu -= device.cpu;
00171             supply.mem -= device.mem;
00172             supply.sto -= device.sto;
00173
00174             return true;
00175         }
00176     };
00177
00181     using Servers = std::vector<Server>;
00182 }
00183
00188 namespace data::generate {
00189
00193     const std::filesystem::path DATA_PATH = std::filesystem::current_path() / "data";
00194
00203     inline std::vector<std::vector<std::string>> services(const int num_services = 10) {
00204         std::filesystem::path service_path = DATA_PATH / "Services.txt";
00205         if (std::filesystem::exists(service_path)) {
00206             return csv::read(service_path.string(),
      ';').value_or(std::vector<std::vector<std::string>>{});
00207         }
00208
00209         std::vector<std::vector<std::string>> services;
00210         services.push_back({"#", "COST", "CPU", "MEM", "STO", "TSK"});
00211
00212         for (int i = 1; i <= num_services; i++) {
00213             int tsk = randomness::random_number(1, 4);
00214             double cpu = 0.0, mem = 0.0, sto = 0.0, cos = 1.0;
00215
00216             for (int j = 0; j < tsk; j++) {
00217                 //cos += randomness::random_number(ZERO, 2.5);
00218                 cpu += randomness::random_number(ZERO, 2.5);
00219                 mem += randomness::random_number(ZERO, 2.5);
00220                 sto += randomness::random_number(ZERO, 15.0);
00221             }
00222
00223             services.push_back({
00224                 convert::to_string(i), convert::to_string(cos, DECIMAL_PLACES),
```

```
00225                 convert::to_string(cpu, DECIMAL_PLACES), convert::to_string(mem, DECIMAL_PLACES),
      convert::to_string(sto, DECIMAL_PLACES),
00226                 convert::to_string(tsk)
00227             });
00228         }
00229
00230         if (!csv::write(service_path.string(), services, ';')) {
00231             return {};
00232         }
00233         return services;
00234     }
00235
00245     inline std::vector<std::vector<std::string>> devices(const int length, const int num_services = 10)
      {
00246         std::filesystem::path device_file_path = DATA_PATH / "devices" / ("Devices_" +
      std::to_string(length) + ".txt");
00247         if (std::filesystem::exists(device_file_path)) {
00248             return csv::read(device_file_path.string(),
      ';').value_or(std::vector<std::vector<std::string>>{});
00249         }
00250
00251         std::vector<std::vector<std::string>> data_services = services(num_services);
00252         if (data_services.size() <= 1) {
00253             return {};
00254         }
00255
00256         std::vector<std::vector<std::string>> devices;
00257         devices.push_back({"#", "COST", "CPU", "MEM", "STO", "TSK", "SVC"});
00258
00259         for (int i = 1; i <= length; i++) {
00260             const auto& source_service_row = data_services.at(randomness::random_number(1,
      static_cast<int>(data_services.size()) - 1));
00261
00262             devices.push_back({
00263                 convert::to_string(i),    // #
00264                 source_service_row.at(1), // COST
00265                 source_service_row.at(2), // CPU
00266                 source_service_row.at(3), // MEM
00267                 source_service_row.at(4), // STO
00268                 source_service_row.at(5), // TASKS
00269                 source_service_row.at(0)  // SERVICE
00270             });
00271         }
00272
00273         if (!csv::write(device_file_path.string(), devices, ';')) {
00274             return {};
00275         }
00276         return devices;
00277     }
00278
00288     inline std::vector<std::vector<std::string>> servers(const int ec_length = 99, const int cc_length
      = 1) {
00289         std::filesystem::path server_file_path = DATA_PATH / "servers" / ("servers_" +
      std::to_string(ec_length) + "_" + std::to_string(cc_length) + ".txt");
00290         if (std::filesystem::exists(server_file_path)) {
00291             return csv::read(server_file_path.string(),
      ';').value_or(std::vector<std::vector<std::string>>{});
00292         }
00293
00294         std::vector<std::vector<std::string>> servers;
00295         servers.push_back({"#", "Type", "CPU", "MEM", "STO"});
00296         int size = ec_length + cc_length;
00297         double cpu, mem, sto;
00298
00299         for (int it = 1; it <= size; it++) {
00300             if (it <= ec_length) {
00301                 cpu = randomness::random_number(2.5, 25.0);
00302                 mem = randomness::random_number(2.5, 125.0);
00303                 sto = randomness::random_number(15.0, 1000.0);
00304             } else {
00305                 double factor = 2.0;
00306                 cpu = randomness::random_number(100000.0/factor, 200000.0/factor);
00307                 mem = randomness::random_number(51200.0/factor, 102800.0/factor);
00308                 sto = randomness::random_number(500000.0/factor, 1000000.0/factor);
00309             }
00310
00311             servers.push_back({
00312                     convert::to_string(it),         // IDX
00313                     (it <= ec_length) ? "E" : "C", // TYPE
00314                     convert::to_string(cpu, DECIMAL_PLACES),        // CPU
00315                     convert::to_string(mem, DECIMAL_PLACES),        // MEM
00316                     convert::to_string(sto, DECIMAL_PLACES)         // STO
00317                 });
00318         }
00319
00320         if (!csv::write(server_file_path.string(), servers, ';')) {
00321             return {};
```

```
00322         }
00323         return servers;
00324     }
00325 }
00326
00331 namespace data::load {
00332
00339     inline core::Devices devices(const int length, const int num_services = 10) {
00340         std::vector<std::vector<std::string» load_devices = data::generate::devices(length,
      num_services);
00341         if (load_devices.size() <= 1) {
00342             std::cerr « "Error: Device data file not found or is empty.\n";
00343             return {};
00344         }
00345
00346         core::Devices devices;
00347         devices.reserve(length + 1);
00348         devices.emplace_back(); // Placeholder for 1-based indexing.
00349
00350         for (const auto& row : load_devices) {
00351             if (row.at(0) == "#") continue;
00352             try {
00353                 devices.emplace_back(
00354                     std::stoi(row.at(0)),  // #
00355                     std::stod(row.at(1)),  // COST
00356                     std::stod(row.at(2)),  // CPU
00357                     std::stod(row.at(3)),  // MEM
00358                     std::stod(row.at(4)),  // STO
00359                     std::stoi(row.at(5)),  // TASKS
00360                     std::stoi(row.at(6))   // SERVICE
00361                 );
00362             } catch (const std::exception& e) {
00363                 std::cerr « "Error parsing device data: " « e.what() « "\n";
00364                 break;
00365             }
00366         }
00367
00368         return devices;
00369     }
00370
00377     inline core::Servers servers(const int ec = 99, const int cc = 1) {
00378         std::vector<std::vector<std::string» load_servers = data::generate::servers(ec, cc);
00379         if (load_servers.size() <= 1) {
00380             std::cerr « "Error: Server data file(s) not found or are empty.\n";
00381             return {};
00382         }
00383
00384         core::Servers servers;
00385         servers.reserve(ec + cc + 1);
00386         servers.emplace_back(); // Placeholder for 1-based indexing.
00387
00388         for (const auto& row : load_servers) {
00389             if (row.at(0) == "#") continue;
00390             try {
00391                 servers.emplace_back(
00392                     std::stoi(row.at(0)),  // #
00393                     row.at(1).front(),     // TYPE (extracting char safely)
00394                     std::stod(row.at(2)),  // CPU
00395                     std::stod(row.at(3)),  // MEM
00396                     std::stod(row.at(4))   // STO
00397                 );
00398             } catch (const std::exception& e) {
00399                 std::cerr « "Error parsing server data: " « e.what() « "\n";
00400                 break;
00401             }
00402         }
00403
00404         return servers;
00405     }
00406 }
00407
00412 namespace metrics {
00413
00417     const std::filesystem::path RESULTS_PATH = std::filesystem::current_path() / "results";
00418
00423     enum class Paradigm {
00424         NONE,
00425         EXACT,
00426         HEURISTIC,
00427         METAHEURISTIC
00428     };
00429
00435     inline std::string to_string(Paradigm paradigm) {
00436         switch (paradigm) {
00437             case Paradigm::EXACT:         return "EXACT";
00438             case Paradigm::HEURISTIC:     return "HEURISTIC";
00439             case Paradigm::METAHEURISTIC: return "METAHEURISTIC";
```

```
00440             default:                        return "NONE";
00441         }
00442     }
00443
00448     enum class Algorithm {
00449         NONE,
00450         solver_maximum_cost,
00451         greedy_cost_benefit,
00452         greedy_random,
00453         greedy_random_fallback,
00454         simulated_annealing
00455     };
00456
00462     inline std::string to_string(Algorithm algorithm) {
00463         switch (algorithm) {
00464             case Algorithm::solver_maximum_cost:    return "SOLVER_MAXIMUM_COST";
00465             case Algorithm::greedy_cost_benefit:    return "GREEDY_CB";
00466             case Algorithm::greedy_random:          return "GREEDY_RANDOM";
00467             case Algorithm::greedy_random_fallback: return "GREEDY_RANDOM_FALLBACK";
00468             case Algorithm::simulated_annealing:    return "SA";
00469             default:                                return "NONE";
00470         }
00471     }
00472
00477     struct MetricEntry {
00478         std::string category;
00479         std::string header;
00480         std::string label;
00481         std::string file_value;
00482         std::string console_value;
00483     };
00484
00489     struct Metrics {
00490         Paradigm paradigm = Paradigm::NONE;
00491         Algorithm algorithm = Algorithm::NONE;
00492         int run_id = 0;
00493
00498         struct Inputs {
00499             int devices_count = 0;
00500             int servers_ec_count = 0;
00501             int servers_cc_count = 0;
00502             double max_penalty_cost = 0;
00503
00504             Inputs& operator=(const Inputs& other) = default;
00505         } inputs;
00506
00511         struct Outputs {
00512             double execution_time = 0.0;
00513             int    devices_served_count = 0;
00514             int    devices_served_ec_count = 0;
00515             int    devices_served_cc_count = 0;
00516             int    servers_used_count = 0;
00517             int    servers_used_ec_count = 0;
00518             int    servers_used_cc_count = 0;
00519             double total_penalty_cost = 0.0;
00520
00521             Outputs& operator=(const Outputs& other) = default;
00522         } outputs;
00523
00530         Metrics(Paradigm paradigm_, Algorithm algorithm_name_, int run_id)
00531             : paradigm(paradigm_), algorithm(algorithm_name_), run_id(run_id) {}
00532
00540         Metrics(Paradigm paradigm_, Algorithm algorithm_name_, int run_id, const
      std::shared_ptr<Metrics>& base)
00541             : paradigm(paradigm_), algorithm(algorithm_name_), run_id(run_id) {
00542                 if (base) inputs = base->inputs;
00543             }
00544
00554         Metrics(Paradigm paradigm_, Algorithm algorithm_name_, int run_id, int d, int s_ec, int s_cc)
00555             : paradigm(paradigm_), algorithm(algorithm_name_), run_id(run_id), inputs({d, s_ec, s_cc,
      0}) {}
00556
00562         Metrics(const std::shared_ptr<Metrics>& base, int run_id)
00563             : paradigm(base ? base->paradigm : Paradigm::NONE),
00564               algorithm(base ? base->algorithm : Algorithm::NONE),
00565               run_id(run_id) {
00566             if (base) {
00567                 inputs = base->inputs;
00568                 outputs = base->outputs;
00569             }
00570         }
00571
00575         virtual ~Metrics() = default;
00576
00581         virtual std::shared_ptr<Metrics> clone() const {
00582             return std::shared_ptr<Metrics>(new Metrics(*this));
00583         }
```

```
00584
00589         virtual std::filesystem::path get_base_directory_path() const {
00590             return RESULTS_PATH / to_string(this->paradigm);
00591         }
00592
00597         virtual std::vector<MetricEntry> build_metrics() const {
00598             int current_total = inputs.devices_count;
00599             int total_servers = inputs.servers_ec_count + inputs.servers_cc_count;
00600
00601             std::vector<MetricEntry> m;
00602
00603             m.push_back({"GENERAL", "ExeTime", "Execution Time (s)",
      convert::to_string(outputs.execution_time, DECIMAL_PLACES), convert::to_string(outputs.execution_time,
      DECIMAL_PLACES)});
00604
00605             m.push_back({"DEVICES", "Devices", "Total", convert::to_string(current_total),
      std::to_string(current_total)});
00606             m.push_back({"DEVICES", "DServed", "Served",
      convert::to_percentage_string(outputs.devices_served_count, current_total),
      std::to_string(outputs.devices_served_count) + " (" +
      convert::to_percentage_string(outputs.devices_served_count, current_total) + "%)"});
00607             m.push_back({"DEVICES", "DServedEC", "  - on EC",
      convert::to_percentage_string(outputs.devices_served_ec_count, outputs.devices_served_count),
      std::to_string(outputs.devices_served_ec_count) + " (" +
      convert::to_percentage_string(outputs.devices_served_ec_count, outputs.devices_served_count) + "% of
      served)"});
00608             m.push_back({"DEVICES", "DServedCC", "  - on CC",
      convert::to_percentage_string(outputs.devices_served_cc_count, outputs.devices_served_count),
      std::to_string(outputs.devices_served_cc_count) + " (" +
      convert::to_percentage_string(outputs.devices_served_cc_count, outputs.devices_served_count) + "% of
      served)"});
00609
00610             m.push_back({"SERVERS", "Servers", "Total", convert::to_string(total_servers),
      std::to_string(total_servers) + " (" + std::to_string(inputs.servers_ec_count) + " EC + " +
      std::to_string(inputs.servers_cc_count) + " CC)"});
00611             m.push_back({"SERVERS", "SUsed", "Used",
      convert::to_percentage_string(outputs.servers_used_count, total_servers),
      std::to_string(outputs.servers_used_count) + " (" +
      convert::to_percentage_string(outputs.servers_used_count, total_servers) + "%)"});
00612             m.push_back({"SERVERS", "SUsedEC", "  - Used EC",
      convert::to_percentage_string(outputs.servers_used_ec_count, inputs.servers_ec_count),
      std::to_string(outputs.servers_used_ec_count) + " (" +
      convert::to_percentage_string(outputs.servers_used_ec_count, inputs.servers_ec_count) + "% of EC)"});
00613             m.push_back({"SERVERS", "SUsedCC", "  - Used CC",
      convert::to_percentage_string(outputs.servers_used_cc_count, inputs.servers_cc_count),
      std::to_string(outputs.servers_used_cc_count) + " (" +
      convert::to_percentage_string(outputs.servers_used_cc_count, inputs.servers_cc_count) + "% of CC)"});
00614
00615             m.push_back({"COSTS", "MaxCost", "MAX PENALTY COST",
      convert::to_string(inputs.max_penalty_cost, DECIMAL_PLACES),
      convert::to_string(inputs.max_penalty_cost, DECIMAL_PLACES)});
00616             m.push_back({"COSTS", "PenaltyCost", "PENALTY COST",
      convert::to_string(outputs.total_penalty_cost, DECIMAL_PLACES),
      convert::to_string(outputs.total_penalty_cost, DECIMAL_PLACES)});
00617
00618             return m;
00619         }
00620
00625         virtual std::vector<std::string> get_header() const {
00626             return {
00627                 "Paradigm", "Algorithm", "Run", "Devices", "Servers_EC", "Servers_CC",
00628                 "Max_cost", "Execution_time", "Devices_served", "Devices_served_ec",
00629                 "Devices_served_cc", "Servers_used", "Servers_used_ec", "Servers_used_cc",
      "Total_cost"
00630             };
00631         }
00632
00637         virtual std::vector<std::string> data() const {
00638             return {
00639                 to_string(paradigm),
00640                 to_string(algorithm),
00641                 std::to_string(run_id),
00642                 std::to_string(inputs.devices_count),
00643                 std::to_string(inputs.servers_ec_count),
00644                 std::to_string(inputs.servers_cc_count),
00645                 convert::to_string(inputs.max_penalty_cost, DECIMAL_PLACES),
00646                 convert::to_string(outputs.execution_time, DECIMAL_PLACES),
00647                 std::to_string(outputs.devices_served_count),
00648                 std::to_string(outputs.devices_served_ec_count),
00649                 std::to_string(outputs.devices_served_cc_count),
00650                 std::to_string(outputs.servers_used_count),
00651                 std::to_string(outputs.servers_used_ec_count),
00652                 std::to_string(outputs.servers_used_cc_count),
00653                 convert::to_string(outputs.total_penalty_cost, DECIMAL_PLACES)
00654             };
00655         }
00656
```

```
00660         inline void save_to_file() const {
00661             std::filesystem::path dir_path = this->get_base_directory_path();
00662             std::filesystem::path result_path = dir_path / (to_string(this->algorithm) + ".txt");
00663 
00664             std::vector<std::vector<std::string» content;
00665             if (!std::filesystem::exists(result_path)) {
00666                 content.push_back(this->get_header());
00667             }
00668             content.push_back(this->data());
00669             file_manager::csv::append(result_path.string(), content, ';');
00670         }
00671     };
00672 
00677     struct ExactMetrics : public Metrics {
00678         std::string status = "Unknown";
00679         double objective_function = 0.0;
00680         double gap = 1.0;
00681 
00688         ExactMetrics(Algorithm algorithm_name_, int run_id_, const std::shared_ptr<Metrics>& base)
00689             : Metrics(Paradigm::EXACT, algorithm_name_, run_id_, base) {
00690                 if (base) inputs = base->inputs;
00691             }
00692 
00701         ExactMetrics(Algorithm algorithm_name_, int run_id, int d, int s_ec, int s_cc) :
00702             Metrics(Paradigm::EXACT, algorithm_name_, run_id, d, s_ec, s_cc) {}
00703 
00709         ExactMetrics(const std::shared_ptr<Metrics>& base, int run_id) :
00710             Metrics(base, run_id) {}
00711 
00716         std::shared_ptr<Metrics> clone() const override {
00717             return std::shared_ptr<ExactMetrics>(new ExactMetrics(*this));
00718         }
00719 
00724         std::vector<MetricEntry> build_metrics() const override {
00725             auto m = Metrics::build_metrics();
00726             m.push_back({"SOLVER STATS", "Status", "Solver Status", status, status});
00727             m.push_back({"SOLVER STATS", "OF", "Objective Function (OF)",
      convert::to_string(objective_function, DECIMAL_PLACES), convert::to_string(objective_function,
      DECIMAL_PLACES)});
00728             m.push_back({"SOLVER STATS", "GAP", "MIP Gap", convert::to_string(gap, DECIMAL_PLACES),
      convert::to_percentage_string(gap, 1.0) + "%"});
00729             return m;
00730         }
00731 
00736         std::vector<std::string> get_header() const override {
00737             auto h = Metrics::get_header();
00738             h.push_back("Status");
00739             h.push_back("Obj_Function");
00740             h.push_back("Gap");
00741             return h;
00742         }
00743 
00748         std::vector<std::string> data() const override {
00749             auto d = Metrics::data();
00750             d.push_back(status);
00751             d.push_back(convert::to_string(objective_function, DECIMAL_PLACES));
00752             d.push_back(convert::to_string(gap, DECIMAL_PLACES));
00753             return d;
00754         }
00755     };
00756 
00761     struct HeuristicMetrics : public Metrics {
00768         HeuristicMetrics(Algorithm algorithm_name_, int run_id_, const std::shared_ptr<Metrics>& base)
      :
00769             Metrics(Paradigm::HEURISTIC, algorithm_name_, run_id_, base) {
00770                 if (base) inputs = base->inputs;
00771             }
00772 
00781         HeuristicMetrics(Algorithm algorithm_name_, int run_id, int d, int s_ec, int s_cc) :
00782             Metrics(Paradigm::HEURISTIC, algorithm_name_, run_id, d, s_ec, s_cc) {}
00783 
00789         HeuristicMetrics(const std::shared_ptr<Metrics>& base, int run_id) :
00790             Metrics(base, run_id) {}
00791 
00796         std::shared_ptr<Metrics> clone() const override {
00797             return std::shared_ptr<HeuristicMetrics>(new HeuristicMetrics(*this));
00798         }
00799     };
00800 
00805     struct MetaHeuristicMetrics : public Metrics {
00806         Algorithm heuristic = Algorithm::NONE;
00807         double temperature = 0.0;
00808         double alpha = 0.0;
00809 
00819         MetaHeuristicMetrics(Algorithm algorithm_name_, int run_id_, const std::shared_ptr<Metrics>&
      base, double temperature_, double alpha_, Algorithm heuristic_)
00820             : Metrics(Paradigm::METAHEURISTIC, algorithm_name_, run_id_, base),
```

```
    temperature(temperature_), alpha(alpha_), heuristic(heuristic_) {
00821                 if (base) inputs = base->inputs;
00822             }
00823
00835         MetaHeuristicMetrics(Algorithm algorithm_name_, int run_id, int d, int s_ec, int s_cc, double
    temperature_, double alpha_, Algorithm heuristic_) :
00836             Metrics(Paradigm::METAHEURISTIC, algorithm_name_, run_id, d, s_ec, s_cc),
    temperature(temperature_), alpha(alpha_), heuristic(heuristic_) {}
00837
00843         MetaHeuristicMetrics(const std::shared_ptr<Metrics>& base, int run_id) :
00844             Metrics(base, run_id) {}
00845
00850         std::shared_ptr<Metrics> clone() const override {
00851             return std::shared_ptr<MetaHeuristicMetrics>(new MetaHeuristicMetrics(*this));
00852         }
00853
00858         std::vector<MetricEntry> build_metrics() const override {
00859             auto m = Metrics::build_metrics();
00860             std::string cat = to_string(algorithm) + " PARAMETERS";
00861             m.push_back({cat, "Heuristic", "Initial Solution", to_string(heuristic),
    to_string(heuristic)});
00862             m.push_back({cat, "Temperature", "Initial Temperature", convert::to_string(temperature,
    DECIMAL_PLACES), convert::to_string(temperature, DECIMAL_PLACES)});
00863             m.push_back({cat, "Alpha", "Alpha (Cooling Rate)", convert::to_string(alpha,
    DECIMAL_PLACES), convert::to_string(alpha, DECIMAL_PLACES)});
00864             return m;
00865         }
00866
00871         std::vector<std::string> get_header() const override {
00872             auto h = Metrics::get_header();
00873             h.push_back("Initial_Heuristic");
00874             h.push_back("Temperature");
00875             h.push_back("Alpha");
00876             return h;
00877         }
00878
00883         std::vector<std::string> data() const override {
00884             auto d = Metrics::data();
00885             d.push_back(to_string(heuristic));
00886             d.push_back(convert::to_string(temperature, DECIMAL_PLACES));
00887             d.push_back(convert::to_string(alpha, DECIMAL_PLACES));
00888             return d;
00889         }
00890     };
00891
00896     struct Result {
00897         core::Devices devices;
00898         core::Servers servers;
00899         std::shared_ptr<Metrics> stats;
00900
00906         Result(core::Devices d, core::Servers s)
00907             : devices(std::move(d)), servers(std::move(s)) {}
00908
00913         Result(const Result& other)
00914             : devices(other.devices), servers(other.servers) {
00915             if (other.stats) {
00916                 stats = other.stats->clone();
00917             }
00918         }
00919
00927         Result(int num_devices, int num_services, int num_ec, int num_cc)
00928             : devices(data::load::devices(num_devices, num_services)),
    servers(data::load::servers(num_ec, num_cc)) {
00929
00930             if (devices.size() <= 1 || servers.size() <= 1) {
00931                 std::cerr << "Error: Data failed to load.\n";
00932                 exit(0);
00933             }
00934
00935             stats = std::make_shared<Metrics>(Paradigm::NONE, Algorithm::NONE, 0);
00936
00937             stats->inputs.devices_count = static_cast<int>(this->devices.size()) - 1;
00938             stats->inputs.servers_ec_count = num_ec;
00939             stats->inputs.servers_cc_count = num_cc;
00940
00941             double total_cost = 0.0;
00942             for (size_t d = 1; d < this->devices.size(); ++d) {
00943                 total_cost += this->devices[d].cost;
00944             }
00945             stats->inputs.max_penalty_cost = static_cast<double>(total_cost);
00946         }
00947
00953         Result& operator=(const Result& other) {
00954             if (this != &other) {
00955                 devices = other.devices;
00956                 servers = other.servers;
00957                 if (other.stats) stats = other.stats->clone();
```

```
00958                 else stats.reset();
00959             }
00960             return *this;
00961         }
00962     };
00963
00968     inline void calculate_metrics(Result& state) {
00969         if (!state.stats) return;
00970
00971         auto& out = state.stats->outputs;
00972
00973         out.devices_served_count = 0;
00974         out.devices_served_ec_count = 0;
00975         out.devices_served_cc_count = 0;
00976         out.servers_used_count = 0;
00977         out.servers_used_ec_count = 0;
00978         out.servers_used_cc_count = 0;
00979         out.total_penalty_cost = 0.0;
00980
00981         for (size_t d = 1; d < state.devices.size(); ++d) {
00982             const auto& device = state.devices[d];
00983             if (device.served) {
00984                 out.devices_served_count++;
00985                 if (device.server_id != -1) {
00986                     if (state.servers[device.server_id].type == 'E') {
00987                         out.devices_served_ec_count++;
00988                     } else {
00989                         out.devices_served_cc_count++;
00990                     }
00991                 }
00992             } else {
00993                 out.total_penalty_cost += device.cost;
00994             }
00995         }
00996
00997         for (size_t s = 1; s < state.servers.size(); ++s) {
00998             const auto& server = state.servers[s];
00999             if (!server.supply.devices.empty()) {
01000                 out.servers_used_count++;
01001                 if (server.type == 'E') {
01002                     out.servers_used_ec_count++;
01003                 } else {
01004                     out.servers_used_cc_count++;
01005                 }
01006             }
01007         }
01008     }
01009
01015     inline void save_results(const Result& state, bool show_servers_usage = false);
01016 }
01017
01022 namespace display {
01023
01028     inline void show_device(const core::Device& device) {
01029         std::cout « "========== Device ID: "  « device.id « " ==========\n"
01030                   « "  - Cost Penalty:       " « device.cost « "\n"
01031                   « "  - Requirements (CPU): " « device.cpu « "\n"
01032                   « "  - Requirements (MEM): " « device.mem « "\n"
01033                   « "  - Requirements (STO): " « device.sto « "\n"
01034                   « "  - Tasks Count:        " « device.tasks « "\n"
01035                   « "  - Service ID:         " « device.service « "\n"
01036                   « "  - State (Served):     " « (device.served ? "Yes" : "No") « "\n";
01037
01038         if (device.served) {
01039             std::cout « "  - Assigned Server ID:   " « device.server_id « "\n";
01040         } else {
01041             std::cout « "  - Assigned Server ID:   None\n";
01042         }
01043         std::cout « "====================================\n" « std::endl;
01044     }
01045
01050     inline void show_devices(const core::Devices& devices) {
01051         std::cout « "\n--- Displaying " « (devices.size() > 0 ? devices.size() - 1 : 0) « " Devices
    ---\n";
01052         for (const auto& device : devices) {
01053             if (device.id == 0) continue;
01054             show_device(device);
01055         }
01056     }
01057
01062     inline void show_server(const core::Server& server) {
01063         std::cout « "========== Server ID: " « server.id « " (Type: " « server.type « ") ==========\n"
01064                   « "  --- Capacity ---\n"
01065                   « "  - Total CPU:           " « server.cpu « "\n"
01066                   « "  - Total MEM:           " « server.mem « "\n"
01067                   « "  - Total STO:           " « server.sto « "\n\n"
01068                   « "  --- Current Demand ---\n"
```

```
01069                   « "  - Used CPU:            " « server.supply.cpu « "\n"
01070                   « "  - Used MEM:            " « server.supply.mem « "\n"
01071                   « "  - Used STO:            " « server.supply.sto « "\n"
01072                   « "  - Devices Served (" « server.supply.devices.size() « "): ";
01073
01074         if (server.supply.devices.empty()) {
01075             std::cout « "None\n";
01076         } else {
01077             std::string device_list = convert::set_to_string(server.supply.devices);
01078             std::cout « device_list « "\n";
01079         }
01080         std::cout « "=============================================\n" « std::endl;
01081     }
01082
01087     inline void show_servers(const core::Servers& servers) {
01088         std::cout « "\n--- Displaying " « (servers.size() > 0 ? servers.size() - 1 : 0) « " Servers
      ---\n";
01089         for (const auto& server : servers) {
01090             if (server.id == 0) continue;
01091             show_server(server);
01092         }
01093     }
01094
01099     namespace details {
01100         const int total_width = 72;
01101         const int label_width = 26;
01102
01109         inline void print_row(std::ostream& out, const std::string& label, const std::string& value) {
01110             out « "| " « std::left « std::setw(label_width) « label
01111                 « " | "« std::left « std::setw(total_width - label_width - 7) « value
01112                 « " |\n";
01113         }
01114
01119         inline void print_header(std::ostream& out) {
01120             out « "+" « std::string(total_width - 2, '=') « "+\n";
01121         }
01122
01127         inline void print_middle(std::ostream& out) {
01128             out « "+" « std::string(label_width + 2, '-') « "+"
01129                 « std::string(total_width - label_width - 5, '-') « "+\n";
01130         }
01131
01137         inline void print_title(std::ostream& out, const std::string& title) {
01138             int padding_total = total_width - 3 - static_cast<int>(title.length());
01139             int padding_left = padding_total / 2;
01140             int padding_right = padding_total - padding_left + 1;
01141             out « "|" « std::string(padding_left, ' ') « title
01142                 « std::string(padding_right, ' ') « "|\n";
01143         }
01144     }
01145
01151     inline void show_metrics_console_or_file(const metrics::Metrics& metrics, std::ostream& out) {
01152         out « "\n";
01153         details::print_header(out);
01154
01155         std::string title = "SIMULATION " + to_string(metrics.algorithm);
01156         details::print_title(out, title);
01157         details::print_middle(out);
01158
01159         details::print_row(out, "Paradigm", to_string(metrics.paradigm));
01160         details::print_row(out, "Algorithm", to_string(metrics.algorithm));
01161         details::print_row(out, "Simulation Run", std::to_string(metrics.run_id));
01162
01163         auto entries = metrics.build_metrics();
01164         std::string current_category = "";
01165
01166         for (const auto& entry : entries) {
01167             if (entry.category != current_category) {
01168                 details::print_middle(out);
01169                 if (entry.category != "GENERAL") {
01170                     details::print_title(out, entry.category);
01171                     details::print_middle(out);
01172                 }
01173                 current_category = entry.category;
01174             }
01175             details::print_row(out, entry.label, entry.console_value);
01176         }
01177         details::print_header(out);
01178     }
01179
01184     inline void show_metrics(const std::shared_ptr<metrics::Metrics>& ptr) {
01185         if (!ptr) return;
01186
01187         show_metrics_console_or_file(*ptr, std::cout);
01188
01189         std::filesystem::path base_dir = ptr->get_base_directory_path();
01190         std::filesystem::create_directories(base_dir);
```

```
01191
01192         std::string config_name = "D" + std::to_string(ptr->inputs.devices_count) +
01193                                   "_S" + std::to_string(ptr->inputs.servers_ec_count +
      ptr->inputs.servers_cc_count);
01194
01195         std::filesystem::path file_path = base_dir / (config_name + "_drawing.txt");
01196
01197         std::ofstream file(file_path, std::ios::app);
01198         if (file.is_open()) {
01199             show_metrics_console_or_file(*ptr, file);
01200         } else {
01201             std::cerr « "Error: Could not save drawing to " « file_path « "\n";
01202         }
01203     }
01204
01209     inline void show_servers_usage(const core::Servers& servers) {
01210         std::cout « "\n+-------------------------------------------------------------+\n";
01211         std::cout « "| SERVERS RESOURCE USAGE                                      |\n";
01212         std::cout « "+-------------------------------------------------------------+\n";
01213
01214         for (const auto& server : servers) {
01215             if (server.id == 0) continue;
01216
01217             std::string cpu_pct = convert::to_percentage_string(server.supply.cpu, server.cpu, 0);
01218             std::string mem_pct = convert::to_percentage_string(server.supply.mem, server.mem, 0);
01219             std::string sto_pct = convert::to_percentage_string(server.supply.sto, server.sto, 0);
01220
01221             std::cout « "  ID: " « std::left « std::setw(4) « server.id
01222                       « "CPU: "  « std::right « std::setw(3) « cpu_pct « "%   "
01223                       « "MEM: "  « std::right « std::setw(3) « mem_pct « "%   "
01224                       « "STO: "  « std::right « std::setw(3) « sto_pct « "%\n";
01225         }
01226         std::cout « "+-------------------------------------------------------------+\n\n";
01227     }
01228 }
01229
01230 inline void metrics::save_results(const Result& state, bool show_servers_usage) {
01231     if (!state.stats) return;
01232     state.stats->save_to_file();
01233     display::show_metrics(state.stats);
01234     if (show_servers_usage) display::show_servers_usage(state.servers);
01235 }
```

7.9 include/tools.hpp File Reference

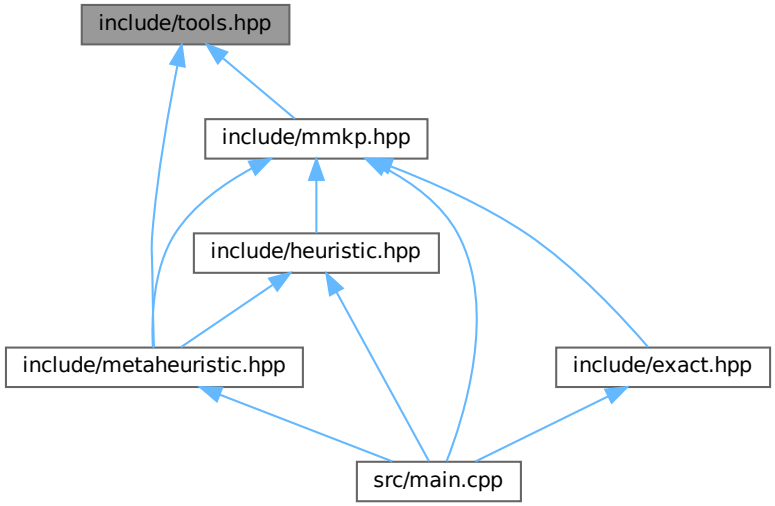
General-purpose utilities for CSV I/O, string formatting, random number generation, and indirect sorting.

```
#include <algorithm>
#include <chrono>
#include <cmath>
#include <exception>
#include <filesystem>
#include <fstream>
#include <iomanip>
#include <iostream>
#include <memory>
#include <numeric>
#include <optional>
#include <random>
#include <set>
#include <sstream>
#include <stdexcept>
#include <string>
#include <type_traits>
#include <utility>
#include <vector>
```

Include dependency graph for tools.hpp:



This graph shows which files directly or indirectly include this file:



Namespaces

- namespace file_manager::csv

  Provides file I/O operations for reading, writing, and appending delimited CSV/text files.
- namespace file_manager
- namespace convert

  Short namespace alias for `file_manager::csv`.
- namespace randomness

  Mersenne Twister (`std::mt19937`) random number generation and permutation utilities.
- namespace sorting

  Generic index-based sorting utilities for ordering 1-indexed entity vectors by member attributes.
- namespace sorting::details

  Internal implementation helpers for pointer-to-member index sorting.

Functions

- std::optional< std::vector< std::vector< std::string > > > file_manager::csv::read (const std::string &file_↩ path, char delimiter=',')

  Reads a delimited CSV file into a 2D vector of strings.
- bool file_manager::csv::write (const std::string &file_path, const std::vector< std::vector< std::string > > &data, char delimiter=',')

Writes a 2D vector of strings to a CSV file, creating parent directories if needed.

- bool file_manager::csv::append (const std::string &file_path, const std::vector< std::vector< std::string > > &data, char delimiter=',')

  Appends a 2D vector of strings to an existing or new CSV file.
- int file_manager::csv::count_lines (const std::string &file_path)

  Counts the total number of lines in a file.
- template<typename T >
  std::string convert::to_string (const T &value, int precision=DECIMAL_PLACES)

  Converts a value of any streamable type to its string representation.
- std::string convert::to_percentage_string (double numerator, double denominator, int precision=DECIMAL_PLACES)

  Calculates a percentage and converts it to a formatted string.
- template<typename T >
  std::string convert::vector_to_string (const std::vector< T > &vec)

  Converts a vector of primitive types to a bracketed array string representation.
- template<typename T >
  std::string convert::set_to_string (const std::set< T > &s)

  Converts a `std::set` of primitive types to a bracketed array string representation.
- std::mt19937 & randomness::get_engine ()

  Provides a singleton instance of a pseudo-random number engine.
- template<typename T >
  T randomness::random_number (T min, T max)

  Generates a uniform random number within a specified inclusive interval $[\text{min}, \text{max}]$.
- template<typename T >
  T randomness::random_number (T min, T max, std::vector< T > &exclude)

  Generates a random number within $[\text{min}, \text{max}]$, excluding values present in `exclude`.
- template<typename T >
  std::vector< T > randomness::shuffled_range (T min, T max)

  Creates a randomly shuffled vector containing all integers in the inclusive range $[\text{min}, \text{max}]$.
- template<bool Ascending, typename T , typename AttributeType , AttributeType T::* AttributePtr>
  void sorting::details::sort_indices_impl (const std::vector< T > &entities, std::vector< int > &idxs)

  Sorts a vector of indices based on a specified struct/class member attribute.
- template<bool Ascending, typename T , typename AttributeType , AttributeType T::* AttributePtr>
  std::vector< int > sorting::sort_entities (const std::vector< T > &entities)

  Sorts the indices of all entities (from `1` to `N-1`) by a specific struct/class attribute.
- template<bool Ascending, typename T , typename AttributeType , AttributeType T::* AttributePtr>
  std::vector< int > sorting::sort_entities (const std::vector< T > &entities, const std::vector< int > &indices↩
  _to_sort)

  Sorts a specific subset of entity indices by a struct/class member attribute.

Variables

- constexpr double ZERO = 1e-13

  Small epsilon constant used for floating-point comparisons and non-zero lower bounds.
- constexpr int DECIMAL_PLACES = 10

  Global default precision (number of decimal places) for floating-point string formatting.

7.9.1 Detailed Description

General-purpose utilities for CSV I/O, string formatting, random number generation, and indirect sorting.

Header-only toolkit providing foundational helper functions used across the MMKP simulator.

7.9.2 Variable Documentation

7.9.2.1 DECIMAL_PLACES

```
constexpr int DECIMAL_PLACES = 10  [constexpr]
```

Global default precision (number of decimal places) for floating-point string formatting.

7.9.2.2 ZERO

```
constexpr double ZERO = 1e-13  [constexpr]
```

Small epsilon constant used for floating-point comparisons and non-zero lower bounds.

7.10 tools.hpp

Go to the documentation of this file.

```
00001
00007 #pragma once
00008
00009 #include <algorithm>
00010 #include <chrono>
00011 #include <cmath>
00012 #include <exception>
00013 #include <filesystem>
00014 #include <fstream>
00015 #include <iomanip>
00016 #include <iostream>
00017 #include <memory>
00018 #include <numeric>
00019 #include <optional>
00020 #include <random>
00021 #include <set>
00022 #include <sstream>
00023 #include <stdexcept>
00024 #include <string>
00025 #include <type_traits>
00026 #include <utility>
00027 #include <vector>
00028
00029
00033 constexpr double ZERO = 1e-13;
00034
00038 constexpr int DECIMAL_PLACES = 10;
00039
00044 namespace file_manager::csv {
00045
00052     inline std::optional<std::vector<std::vector<std::string>>> read(const std::string& file_path, char
      delimiter = ',') {
00053         std::ifstream file(file_path);
00054         if (!file.is_open()) {
00055             std::cerr << "Error: Could not open file " << file_path << "\n";
00056             return std::nullopt;
00057         }
00058
00059         std::vector<std::vector<std::string>> file_data;
00060         std::string line;
00061
00062         while (std::getline(file, line)) {
00063             std::vector<std::string> row;
00064             std::stringstream ss(line);
00065             std::string cell;
00066
00067             while (std::getline(ss, cell, delimiter)) {
00068                 row.push_back(cell);
00069             }
00070
00071             if (!row.empty()) {
00072                 file_data.push_back(row);
00073             }
00074         }
```

```
00075         return file_data;
00076     }
00077
00085     inline bool write(const std::string& file_path, const std::vector<std::vector<std::string>& data,
      char delimiter = ',') {
00086         std::filesystem::path path(file_path);
00087         if (path.has_parent_path()) {
00088             std::filesystem::create_directories(path.parent_path());
00089         }
00090
00091         std::ofstream file(file_path);
00092         if (!file.is_open()) {
00093             std::cerr << "Error: Could not create or open file " << file_path << "\n";
00094             return false;
00095         }
00096
00097         for (size_t i = 0; i < data.size(); ++i) {
00098             for (size_t j = 0; j < data[i].size(); ++j) {
00099                 file << data[i][j];
00100                 if (j < data[i].size() - 1) {
00101                     file << delimiter;
00102                 }
00103             }
00104             if (i < data.size() - 1) {
00105                 file << "\n";
00106             }
00107         }
00108         return true;
00109     }
00110
00118     inline bool append(const std::string& file_path, const std::vector<std::vector<std::string>& data,
      char delimiter = ',') {
00119         std::filesystem::path path(file_path);
00120         if (path.has_parent_path()) {
00121             std::filesystem::create_directories(path.parent_path());
00122         }
00123
00124         std::ofstream file(file_path, std::ios::app);
00125         if (!file.is_open()) {
00126             std::cerr << "Error: Could not create or open file for appending " << file_path << "\n";
00127             return false;
00128         }
00129
00130         if (file.tellp() > 0) {
00131             file << "\n";
00132         }
00133
00134         for (size_t i = 0; i < data.size(); ++i) {
00135             for (size_t j = 0; j < data[i].size(); ++j) {
00136                 file << data[i][j];
00137                 if (j < data[i].size() - 1) {
00138                     file << delimiter;
00139                 }
00140             }
00141             if (i < data.size() - 1) {
00142                 file << "\n";
00143             }
00144         }
00145         return true;
00146     }
00147
00153     inline int count_lines(const std::string& file_path) {
00154         if (!std::filesystem::exists(file_path)) {
00155             return 0;
00156         }
00157
00158         std::ifstream file(file_path);
00159         if (!file.is_open()) {
00160             return 0;
00161         }
00162
00163         int count = 0;
00164         std::string line;
00165         while (std::getline(file, line)) {
00166             count++;
00167         }
00168         return count;
00169     }
00170
00171 }
00172
00176 namespace csv = file_manager::csv;
00177
00182 namespace convert {
00183
00192     template <typename T>
00193     inline std::string to_string(const T& value, int precision = DECIMAL_PLACES) {
```

```
00194         std::ostringstream out;
00195         if constexpr (std::is_floating_point_v<T>) {
00196             out « std::fixed « std::setprecision(precision);
00197         }
00198         out « value;
00199         return out.str();
00200     }
00201
00210     inline std::string to_percentage_string(double numerator, double denominator, int precision =
      DECIMAL_PLACES) {
00211         if (denominator == 0.0) {
00212             return to_string(0.0, precision);
00213         }
00214         double percentage = (numerator / denominator) * 100.0;
00215         return to_string(percentage, precision);
00216     }
00217
00224     template <typename T>
00225     inline std::string vector_to_string(const std::vector<T>& vec) {
00226         if (vec.empty()) return "[]";
00227         std::string str = "[";
00228         for (size_t i = 0; i < vec.size(); ++i) {
00229             str += std::to_string(vec[i]);
00230             if (i < vec.size() - 1) str += ",";
00231         }
00232         str += "]";
00233         return str;
00234     }
00235
00242     template <typename T>
00243     inline std::string set_to_string(const std::set<T>& s) {
00244         if (s.empty()) return "[]";
00245         std::string str = "[";
00246         auto it = s.begin();
00247         while (it != s.end()) {
00248             str += std::to_string(*it);
00249             ++it;
00250             if (it != s.end()) str += ",";
00251         }
00252         str += "]";
00253         return str;
00254     }
00255 }
00256
00261 namespace randomness {
00262
00268     inline std::mt19937& get_engine() {
00269         static std::random_device rd;
00270         static std::mt19937 engine(rd());
00271         return engine;
00272     }
00273
00282     template<typename T>
00283     inline T random_number(T min, T max) {
00284         static_assert(std::is_arithmetic_v<T>, "T must be an arithmetic type.");
00285         if (min > max) {
00286             throw std::invalid_argument("Error in random_number: min cannot be greater than max.");
00287         }
00288
00289         if constexpr (std::is_integral_v<T>) {
00290             std::uniform_int_distribution<T> distribution(min, max);
00291             return distribution(get_engine());
00292         } else {
00293             std::uniform_real_distribution<T> distribution(min, max);
00294             return distribution(get_engine());
00295         }
00296     }
00297
00308     template<typename T>
00309     inline T random_number(T min, T max, std::vector<T>& exclude) {
00310         static_assert(std::is_arithmetic_v<T>, "T must be an arithmetic type.");
00311
00312         if (min > max) {
00313             throw std::invalid_argument("Error in random_number: min cannot be greater than max.");
00314         }
00315
00316         if constexpr (std::is_integral_v<T>) {
00317             if (exclude.size() >= static_cast<size_t>(max - min + 1)) {
00318                 throw std::invalid_argument("Error in random_number: all possible values are excluded
      causing infinite loop.");
00319             }
00320         }
00321
00322         T result;
00323         if constexpr (std::is_integral_v<T>) {
00324             std::uniform_int_distribution<T> distribution(min, max);
00325             do {
```

```
00326                 result = distribution(get_engine());
00327             } while (std::find(exclude.begin(), exclude.end(), result) != exclude.end());
00328         } else {
00329             std::uniform_real_distribution<T> distribution(min, max);
00330             do {
00331                 result = distribution(get_engine());
00332             } while (std::find(exclude.begin(), exclude.end(), result) != exclude.end());
00333         }
00334
00335         exclude.push_back(result);
00336         return result;
00337     }
00338
00347     template<typename T>
00348     inline std::vector<T> shuffled_range(T min, T max) {
00349         static_assert(std::is_integral_v<T>, "shuffled_range requires an integral type.");
00350         if (min > max) {
00351             throw std::invalid_argument("Error in shuffled_range: min cannot be greater than max.");
00352         }
00353         std::vector<T> numbers(max - min + 1);
00354         std::iota(numbers.begin(), numbers.end(), min);
00355         std::shuffle(numbers.begin(), numbers.end(), get_engine());
00356         return numbers;
00357     }
00358 }
00359
00364 namespace sorting {
00365
00370     namespace details {
00371
00381         template <bool Ascending, typename T, typename AttributeType, AttributeType T::* AttributePtr>
00382         void sort_indices_impl(const std::vector<T>& entities, std::vector<int>& idxs) {
00383             std::sort(idxs.begin(), idxs.end(), [&](int i, int j) {
00384                 const auto& attr_i = entities.at(i).*AttributePtr;
00385                 const auto& attr_j = entities.at(j).*AttributePtr;
00386
00387                 if (attr_i != attr_j) {
00388                     if constexpr (Ascending) {
00389                         return attr_i < attr_j; // Ascending order
00390                     } else {
00391                         return attr_i > attr_j; // Descending order
00392                     }
00393                 }
00394                 return i < j; // Tie-breaker
00395             });
00396         }
00397     }
00398
00409     template <bool Ascending, typename T, typename AttributeType, AttributeType T::* AttributePtr>
00410     inline std::vector<int> sort_entities(const std::vector<T>& entities) {
00411         if (entities.size() <= 1) return {};
00412         std::vector<int> idxs(entities.size() - 1);
00413         std::iota(idxs.begin(), idxs.end(), 1); // Assumes 1-based indexing
00414         details::sort_indices_impl<Ascending, T, AttributeType, AttributePtr>(entities, idxs);
00415         return idxs;
00416     }
00417
00428     template <bool Ascending, typename T, typename AttributeType, AttributeType T::* AttributePtr>
00429     inline std::vector<int> sort_entities(const std::vector<T>& entities, const std::vector<int>&
      indices_to_sort) {
00430         if (entities.empty() || indices_to_sort.empty()) return {};
00431         std::vector<int> idxs = indices_to_sort; // Create a copy to sort
00432         details::sort_indices_impl<Ascending, T, AttributeType, AttributePtr>(entities, idxs);
00433         return idxs;
00434     }
00435 }
```

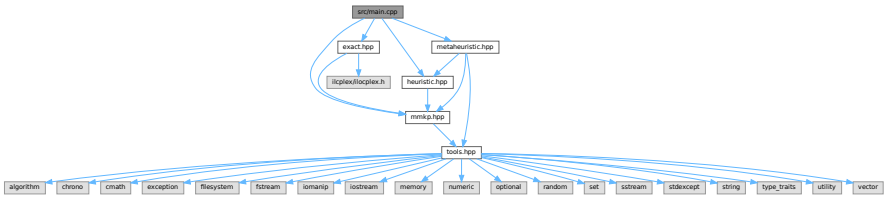
7.11 src/main.cpp File Reference

Entry point and benchmark runner for the MMKP IoT-EC-CC resource allocation simulator.

```
#include "mmkp.hpp"
#include "exact.hpp"
#include "heuristic.hpp"
```

```
#include "metaheuristic.hpp"
```

Include dependency graph for main.cpp:



Functions

- void run_simulation (const metrics::Result &base, int option, int run_id)

  Executes a specific algorithm configuration for the network resource allocation simulation.
- int main ()

  Main entry point for the network resource allocation simulation benchmark.

Variables

- constexpr int SERVICES = 100

  Total number of distinct service profiles generated.
- constexpr int DEVICES_START = 10000

  Initial number of IoT devices in the scaling benchmark.
- constexpr int DEVICES_STEP = 1000

  Increment step for the number of IoT devices per batch.
- constexpr int DEVICES_END = 20000

  Maximum number of IoT devices in the scaling benchmark.
- constexpr int EC_LENGTH = 499

  Total number of Edge Computing (EC) servers available.
- constexpr int CC_LENGTH = 1

  Total number of Cloud Computing (CC) servers available.
- constexpr int MAX_RUNS = 50

  Number of independent runs per instance configuration.
- constexpr double CPLEX_TIME_LIMIT = 3600

  Time limit in seconds for the CPLEX exact solver.
- constexpr double TEMPERATURE = 50.0

  Initial temperature for Simulated Annealing.
- constexpr double ALPHA = 0.9999

  Geometric cooling factor for Simulated Annealing.

7.11.1 Detailed Description

Entry point and benchmark runner for the MMKP IoT-EC-CC resource allocation simulator.

Defines the experimental parameters (scaling of IoT devices, server counts, CPLEX time limit, and Simulated Annealing cooling schedule) and orchestrates the execution across Exact, Heuristic, and Meta-heuristic paradigms.

7.11.2 Function Documentation

7.11.2.1 main()

```
int main ( )
```

Main entry point for the network resource allocation simulation benchmark.

Iterates over the configured range of device counts and independent runs, executing deterministic algorithms once per instance and stochastic algorithms across all runs.

Returns

int Exit status code (`0` on normal termination).

7.11.2.2 run_simulation()

```
void run_simulation (
            const metrics::Result & base,
            int option,
            int run_id )
```

Executes a specific algorithm configuration for the network resource allocation simulation.

Instantiates the appropriate metrics tracker based on the selected option, dispatches the simulation state to the corresponding solver paradigm (Exact, Heuristic, or Meta-heuristic), computes final utilization/cost metrics, and saves the results to disk.

Parameters

base	The initial system state containing untouched devices and servers.
option	The identifier for the algorithm combination to execute: • `0`: Exact CPLEX solver (`solver_maximum_cost`) • `-1`: Constructive heuristic (`greedy_cost_benefit`) • `-2`: Constructive heuristic (`greedy_random`) • `-3`: Constructive heuristic (`greedy_random_fallback`) • `1`: Simulated Annealing + `greedy_cost_benefit` • `2`: Simulated Annealing + `greedy_random` • `3`: Simulated Annealing + `greedy_random_fallback`
run_id	The current execution run number (1-indexed).

7.11.3 Variable Documentation

7.11.3.1 ALPHA

```
constexpr double ALPHA = 0.9999  [constexpr]
```

Geometric cooling factor for Simulated Annealing.

7.11.3.2 CC_LENGTH

```
constexpr int CC_LENGTH = 1  [constexpr]
```

Total number of Cloud Computing (CC) servers available.

7.11.3.3 CPLEX_TIME_LIMIT

```
constexpr double CPLEX_TIME_LIMIT = 3600  [constexpr]
```

Time limit in seconds for the CPLEX exact solver.

7.11.3.4 DEVICES_END

```
constexpr int DEVICES_END = 20000  [constexpr]
```

Maximum number of IoT devices in the scaling benchmark.

7.11.3.5 DEVICES_START

```
constexpr int DEVICES_START = 10000  [constexpr]
```

Initial number of IoT devices in the scaling benchmark.

7.11.3.6 DEVICES_STEP

```
constexpr int DEVICES_STEP = 1000  [constexpr]
```

Increment step for the number of IoT devices per batch.

7.11.3.7 EC_LENGTH

```
constexpr int EC_LENGTH = 499  [constexpr]
```

Total number of Edge Computing (EC) servers available.

7.11.3.8 MAX_RUNS

```
constexpr int MAX_RUNS = 50  [constexpr]
```

Number of independent runs per instance configuration.

7.11.3.9 SERVICES

```
constexpr int SERVICES = 100  [constexpr]
```

Total number of distinct service profiles generated.

7.11.3.10 TEMPERATURE

```
constexpr double TEMPERATURE = 50.0  [constexpr]
```

Initial temperature for Simulated Annealing.

Index

~Metrics
  metrics::Metrics, 59

add_service
  core::Server, 69
Algorithm
  metrics, 28
algorithm
  metrics::Metrics, 60
ALPHA
  main.cpp, 107
alpha
  metrics::MetaHeuristicMetrics, 54
append
  file_manager::csv, 22
apply_neighbor
  metaheuristic::anonymous_namespace{metaheuristic.hpp}, 26

boot
  exact, 20
  heuristic, 23
  metaheuristic, 25
build_metrics
  metrics::ExactMetrics, 43
  metrics::MetaHeuristicMetrics, 53
  metrics::Metrics, 59

calculate_metrics
  metrics, 29
can_serve
  core::Server, 69
category
  metrics::MetricEntry, 55
CC_LENGTH
  main.cpp, 107
clone
  metrics::ExactMetrics, 43
  metrics::HeuristicMetrics, 47
  metrics::MetaHeuristicMetrics, 53
  metrics::Metrics, 59
console_value
  metrics::MetricEntry, 55
convert, 9
  set_to_string, 9
  to_percentage_string, 10
  to_string, 10
  vector_to_string, 11
core, 11
  Devices, 12
  Servers, 12
core::Device, 37
  cost, 38
  cpu, 38
  Device, 38
  id, 39
  mem, 39
  served, 39
  server_id, 39
  service, 39
  sto, 39
  tasks, 39
core::Server, 67
  add_service, 69
  can_serve, 69
  cpu, 70
  id, 70
  mem, 70
  rmv_service, 69
  Server, 68
  sto, 70
  supply, 70
  type, 70
core::Supply, 70
  cost, 72
  cpu, 72
  devices, 72
  mem, 72
  sto, 72
  Supply, 71
cost
  core::Device, 38
  core::Supply, 72
count_lines
  file_manager::csv, 22
CPLEX_TIME_LIMIT
  main.cpp, 107
cpu
  core::Device, 38
  core::Server, 70
  core::Supply, 72

data, 12
  metrics::ExactMetrics, 43
  metrics::MetaHeuristicMetrics, 53
  metrics::Metrics, 59
data::generate, 12
  DATA_PATH, 14
  devices, 13
  servers, 13

services, 13
data::load, 14
devices, 14
servers, 15
DATA_PATH
data::generate, 14
DECIMAL_PLACES
tools.hpp, 101
delta_cost
metaheuristic::anonymous_namespace{metaheuristic.hpp}::Move, 62
Device
core::Device, 38
device_id
metaheuristic::anonymous_namespace{metaheuristic.hpp}::Move, 62
Devices
core, 12
devices
core::Supply, 72
data::generate, 13
data::load, 14
metrics::Result, 66
devices_count
metrics::Metrics::Inputs, 49
DEVICES_END
main.cpp, 107
devices_served_cc_count
metrics::Metrics::Outputs, 63
devices_served_count
metrics::Metrics::Outputs, 63
devices_served_ec_count
metrics::Metrics::Outputs, 63
DEVICES_START
main.cpp, 107
DEVICES_STEP
main.cpp, 107
display, 15
show_device, 16
show_devices, 16
show_metrics, 16
show_metrics_console_or_file, 16
show_server, 17
show_servers, 17
show_servers_usage, 17
display::details, 17
label_width, 19
print_header, 18
print_middle, 18
print_row, 18
print_title, 19
total_width, 19

EC_LENGTH
main.cpp, 107
EXACT
metrics, 29
exact, 19
boot, 20
NumVar2D, 20
exact::anonymous_namespace{exact.hpp}, 21
solver_maximum_cost, 21
ExactMetrics
metrics::ExactMetrics, 42, 43
execution_time
metrics::Metrics::Outputs, 64

file_manager, 21
file_manager::csv, 21
append, 22
count_lines, 22
read, 22
write, 23
file_value
metrics::MetricEntry, 55

gap
metrics::ExactMetrics, 44
get_base_directory_path
metrics::Metrics, 59
get_engine
randomness, 31
get_header
metrics::ExactMetrics, 44
metrics::MetaHeuristicMetrics, 53
metrics::Metrics, 60
greedy_cost_benefit
heuristic::anonymous_namespace{heuristic.hpp}, 24
metrics, 29
greedy_random
heuristic::anonymous_namespace{heuristic.hpp}, 24
metrics, 29
greedy_random_fallback
heuristic::anonymous_namespace{heuristic.hpp}, 25
metrics, 29

header
metrics::MetricEntry, 55
HEURISTIC
metrics, 29
heuristic, 23
boot, 23
metrics::MetaHeuristicMetrics, 54
heuristic::anonymous_namespace{heuristic.hpp}, 24
greedy_cost_benefit, 24
greedy_random, 24
greedy_random_fallback, 25
HeuristicMetrics
metrics::HeuristicMetrics, 46, 47

i_matrix
mmkp.hpp, 87
i_set
mmkp.hpp, 87
i_vector

mmkp.hpp, 87
id
core::Device, 39
core::Server, 70
id_pair
mmkp.hpp, 87
id_pair_vector
mmkp.hpp, 87
ii_pair
mmkp.hpp, 87
ii_pair_vector
mmkp.hpp, 87
include/exact.hpp, 73, 74
include/heuristic.hpp, 76, 77
include/metaheuristic.hpp, 79, 81
include/mmkp.hpp, 83, 88
include/tools.hpp, 98, 101
inputs
metrics::Metrics, 60

label
metrics::MetricEntry, 55
label_width
display::details, 19
LOOP
metaheuristic::anonymous_namespace{metaheuristic.hpp}, 27

main
main.cpp, 106
main.cpp
ALPHA, 107
CC_LENGTH, 107
CPLEX_TIME_LIMIT, 107
DEVICES_END, 107
DEVICES_START, 107
DEVICES_STEP, 107
EC_LENGTH, 107
main, 106
MAX_RUNS, 107
run_simulation, 106
SERVICES, 108
TEMPERATURE, 108
max_penalty_cost
metrics::Metrics::Inputs, 49
MAX_RUNS
main.cpp, 107
mem
core::Device, 39
core::Server, 70
core::Supply, 72
METAHEURISTIC
metrics, 29
metaheuristic, 25
boot, 25
metaheuristic::anonymous_namespace{metaheuristic.hpp}, 26
apply_neighbor, 26
LOOP, 27
revert_neighbor, 27
simulated_annealing, 27
metaheuristic::anonymous_namespace{metaheuristic.hpp}::Move, 61
delta_cost, 62
device_id, 62
new_server_id, 62
old_server_id, 62
swapped_id, 62
valid, 62
MetaHeuristicMetrics
metrics::MetaHeuristicMetrics, 51, 52
Metrics
metrics::Metrics, 57, 58
metrics, 27
Algorithm, 28
calculate_metrics, 29
EXACT, 29
greedy_cost_benefit, 29
greedy_random, 29
greedy_random_fallback, 29
HEURISTIC, 29
METAHEURISTIC, 29
NONE, 29
Paradigm, 29
RESULTS_PATH, 30
save_results, 29
simulated_annealing, 29
solver_maximum_cost, 29
to_string, 30
metrics::ExactMetrics, 40
build_metrics, 43
clone, 43
data, 43
ExactMetrics, 42, 43
gap, 44
get_header, 44
objective_function, 44
status, 44
metrics::HeuristicMetrics, 45
clone, 47
HeuristicMetrics, 46, 47
metrics::MetaHeuristicMetrics, 49
alpha, 54
build_metrics, 53
clone, 53
data, 53
get_header, 53
heuristic, 54
MetaHeuristicMetrics, 51, 52
temperature, 54
metrics::MetricEntry, 54
category, 55
console_value, 55
file_value, 55
header, 55
label, 55
metrics::Metrics, 56

~Metrics, 59
algorithm, 60
build_metrics, 59
clone, 59
data, 59
get_base_directory_path, 59
get_header, 60
inputs, 60
Metrics, 57, 58
outputs, 60
paradigm, 60
run_id, 61
save_to_file, 60
metrics::Metrics::Inputs, 48
devices_count, 49
max_penalty_cost, 49
operator=, 48
servers_cc_count, 49
servers_ec_count, 49
metrics::Metrics::Outputs, 62
devices_served_cc_count, 63
devices_served_count, 63
devices_served_ec_count, 63
execution_time, 64
operator=, 63
servers_used_cc_count, 64
servers_used_count, 64
servers_used_ec_count, 64
total_penalty_cost, 64
metrics::Result, 64
devices, 66
operator=, 66
Result, 65, 66
servers, 66
stats, 67
mmkp.hpp
i_matrix, 87
i_set, 87
i_vector, 87
id_pair, 87
id_pair_vector, 87
ii_pair, 87
ii_pair_vector, 87
s_matrix, 88
s_vector, 88

new_server_id
metaheuristic::anonymous_namespace{metaheuristic.hpp}::Move, 62
NONE
metrics, 29
NumVar2D
exact, 20

objective_function
metrics::ExactMetrics, 44
old_server_id
metaheuristic::anonymous_namespace{metaheuristic.hpp}::Move, 62
operator=
metrics::Metrics::Inputs, 48
metrics::Metrics::Outputs, 63
metrics::Result, 66
outputs
metrics::Metrics, 60

Paradigm
metrics, 29
paradigm
metrics::Metrics, 60
print_header
display::details, 18
print_middle
display::details, 18
print_row
display::details, 18
print_title
display::details, 19

random_number
randomness, 31, 32
randomness, 30
get_engine, 31
random_number, 31, 32
shuffled_range, 32
read
file_manager::csv, 22
Result
metrics::Result, 65, 66
RESULTS_PATH
metrics, 30
revert_neighbor
metaheuristic::anonymous_namespace{metaheuristic.hpp}, 27
rmv_service
core::Server, 69
run_id
metrics::Metrics, 61
run_simulation
main.cpp, 106

s_matrix
mmkp.hpp, 88
s_vector
mmkp.hpp, 88
save_results
metrics, 29
save_to_file
metrics::Metrics, 60
served
core::Device, 39
Server
core::Server, 68
server_id
core::Device, 39
Servers
core, 12
servers

data::generate, 13
data::load, 15
metrics::Result, 66
servers_cc_count
metrics::Metrics::Inputs, 49
servers_ec_count
metrics::Metrics::Inputs, 49
servers_used_cc_count
metrics::Metrics::Outputs, 64
servers_used_count
metrics::Metrics::Outputs, 64
servers_used_ec_count
metrics::Metrics::Outputs, 64
service
core::Device, 39
SERVICES
main.cpp, 108
services
data::generate, 13
set_to_string
convert, 9
show_device
display, 16
show_devices
display, 16
show_metrics
display, 16
show_metrics_console_or_file
display, 16
show_server
display, 17
show_servers
display, 17
show_servers_usage
display, 17
shuffled_range
randomness, 32
simulated_annealing
metaheuristic::anonymous_namespace{metaheuristic.hpp}, 27
metrics, 29
solver_maximum_cost
exact::anonymous_namespace{exact.hpp}, 21
metrics, 29
sort_entities
sorting, 34
sort_indices_impl
sorting::details, 35
sorting, 33
sort_entities, 34
sorting::details, 35
sort_indices_impl, 35
src/main.cpp, 104
stats
metrics::Result, 67
status
metrics::ExactMetrics, 44
sto
core::Device, 39
core::Server, 70
core::Supply, 72
Supply
core::Supply, 71
supply
core::Server, 70
swapped_id
metaheuristic::anonymous_namespace{metaheuristic.hpp}::Move, 62

tasks
core::Device, 39
TEMPERATURE
main.cpp, 108
temperature
metrics::MetaHeuristicMetrics, 54
to_percentage_string
convert, 10
to_string
convert, 10
metrics, 30
tools.hpp
DECIMAL_PLACES, 101
ZERO, 101
total_penalty_cost
metrics::Metrics::Outputs, 64
total_width
display::details, 19
type
core::Server, 70

valid
metaheuristic::anonymous_namespace{metaheuristic.hpp}::Move, 62
vector_to_string
convert, 11

write
file_manager::csv, 23

ZERO
tools.hpp, 101